

POLITECNICO
MILANO 1863

Requirement Analysis and Specification Document

Deliverable:	RASD
Title:	Requirement Analysis and Specification Document
Authors:	Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo
Version:	1.0
Date:	23-December-2025
Download page:	https://github.com/nicolasbolanospolimi/BarberaBolanosNuzzo
Copyright:	Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Contents

Table of Contents . . . 3

List of Figures . . . 5

List of Tables . . . 5

1 Introduction . . . 6
- 1.1 Purpose . . . 6
  - 1.1.1 Goals . . . 6
- 1.2 Scope . . . 6
  - 1.2.1 World Phenomena . . . 7
  - 1.2.2 Shared Phenomena . . . 7
- 1.3 Definitions, Acronyms, Abbreviations . . . 7
  - 1.3.1 Definitions . . . 7
  - 1.3.2 Acronyms . . . 8
  - 1.3.3 Abbreviations . . . 8
- 1.4 Revision History . . . 8
- 1.5 Reference Documents . . . 9
- 1.6 Document Structure . . . 9

2 Overall Description . . . 10
- 2.1 Product perspective . . . 10
  - 2.1.1 Scenarios . . . 10
  - 2.1.2 Domain Class Diagrams . . . 12
  - 2.1.3 State Machine Diagrams . . . 14
- 2.2 Product functions . . . 17
- 2.3 Users characteristics . . . 19
  - 2.3.1 Any user (non-registered and registered users) . . . 19
  - 2.3.2 Registered users . . . 19
- 2.4 Assumptions, dependencies and constraints . . . 20
  - 2.4.1 Regulatory policies . . . 20
  - 2.4.2 Domain Assumptions . . . 20

3 Specific Requirements . . . 21
- 3.1 External Interface Requirements . . . 21
  - 3.1.1 User Interfaces . . . 21
  - 3.1.2 Hardware Interfaces . . . 26
  - 3.1.3 Software Interfaces . . . 27
  - 3.1.4 Communication Interfaces . . . 27
- 3.2 Functional Requirements . . . 28
  - 3.2.1 Use Case Diagram [5] . . . 31
  - 3.2.2 Use Cases . . . 32
  - 3.2.3 Mapping . . . 38
  - 3.2.4 Sequence Diagrams [4] . . . 44
- 3.3 Performance Requirements . . . 57
- 3.4 Design Constraints . . . 57
  - 3.4.1 Standards compliance . . . 57
  - 3.4.2 Hardware Limitation . . . 57
  - 3.4.3 Any other constraint . . . 58
- 3.5 Software System Attribute . . . 58
  - 3.5.1 Scalability . . . 58
  - 3.5.2 Availability . . . 58
  - 3.5.3 Reliability . . . 59
  - 3.5.4 Security and privacy . . . 59
  - 3.5.5 Maintainability . . . 59
  - 3.5.6 Portability . . . 59

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

4 Formal Analysis Using Alloy **60**
4.1 Alloy code 60

5 Effort Spent **67**

6 Use of Generative AI **67**
6.1 Use of Generative Artificial Intelligence Tools 67

References **69**

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

List of Figures

1 Domain class diagram for BBP. . . . 12
2 Sign in page of BBP. . . . 21
3 Sign up page of BBP. . . . 21
4 Possible screen visualization after a search in BBP. Note: If a User is not registered, the navigation bar is not present . . . 22
5 Home page of BBP. . . . 23
6 List of trips registered (not published paths). . . . 23
7 Recording a new activity using BBP. . . . 24
8 Reviewing your recorded activities using BBP. . . . 24
9 Page used to insert tracked paths to be published and made available to every user. . . . 25
10 After doing an automated tracking of a path, this is the first screen the user sees when booting up the app. . . . 26
11 Reviewing the automated tracking paths before publishing them. . . . 26
12 Use Case Diagram . . . 31
13 Sequence diagram for register. . . . 45
14 Sequence diagram for find best path. . . . 46
15 Sequence diagram for login. . . . 47
16 Sequence diagram for record trip. . . . 48
17 Sequence diagram for attach weather information. . . . 49
18 Sequence diagram for review trip. . . . 50
19 Sequence diagram for enable automated mode. . . . 51
20 Sequence diagram for disable automated mode. . . . 52
21 Sequence diagram for record a path in manual mode. . . . 53
22 Sequence diagram for record a path in automated mode. . . . 54
23 Sequence diagram for publish a pre-recorded automated mode path. . . . 55
24 Sequence diagram for logout. . . . 56
25 Idle phase . . . 65
26 Select Phase . . . 65
27 Ride phase . . . 65
28 Checkinfo phase . . . 65
29 Publish phase . . . 66

List of Tables

1 Organization Registration Use Case . . . 32
2 Find the best path Use Case . . . 33
3 Login Use Case . . . 33
4 Record a trip Use Case . . . 34
5 Attach weather information Use Case . . . 34
6 Review a trip Use Case . . . 35
7 Record a Path in Automated Mode Use Case . . . 35
8 Enable Automated Mode Use Case . . . 36
9 Disable Automated Mode Use Case . . . 36
10 Publish a pre-recorded automated mode path Use Case . . . 37
11 Record a path in manual mode Use Case . . . 37
12 Logout Use Case . . . 38

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

1 Introduction

1.1 Purpose

Cycling is not always the safest activity, the cyclist does not always have a dedicated infrastructure and sometimes must share among other road actors. Also, people are not always aware of the existence of paths, or roads that are in better condition than other when going from one place to another. In order to make biking a more efficient and safe, both as a sporting activity and mean of transport, Best Bike Paths (BBP) is created.

BBP aims to promote safer, more efficient, and enjoyable cycling by enabling users to discover and evaluate bike routes based on path conditions, obstacles, and user feedback. BBP supports active contribution and collaboration from cyclists, allowing them to record their trips, share path conditions, and confirm or correct automatically collected data, fostering a community-driven platform that keeps information current and reliable.

BBP encourages sustainable urban mobility by motivating more people to cycle through improved route transparency and safety awareness. It also helps local authorities and communities by aggregating data to identify needed maintenance or hazards on bike paths, potentially contributing to better infrastructure planning and maintenance.

1.1.1 Goals

G1: Users can register and have a profile in BBP application.
G2: Users manually create bike paths in an inventory by specifying the name of the streets and the status.
G3: Users create bike paths in an automated mode where the system acquire data from the user's mobile device and reconstruct the path.
G4: Users browse bike paths within the inventory. They can specify an origin and destination and ask the system to visualize the bike path on a map.
G5: Users record their trips and store them to keep track of their cycling activities.
G6: Users can see for a recorded trip various statistical data of the user performances as well as meteorological information during the respective trip.
G7: The system uses information published by the users about the paths by merging pieces of information to generate the optimal path, using the freshness and number of confirmed data.

1.2 Scope

The core goal is to help cyclists easily discover and evaluate paths based on user-contributed trip data, path conditions, and obstacles. BBP enables both registered and unregistered users to explore and access reliable data about cycling paths based on their quality, safety, and efficiency.

BBP allows users to contribute information about bike paths either manually or automatically. In manual mode, users enter data such as street names, surface conditions, and obstacles. In automatic mode, BBP leverages mobile device sensors, including GPS, accelerometers, and gyroscopes, to detect paths and irregularities during rides. The system filters these inputs for accuracy through user confirmation before making them public.

BBP also aggregates and merges data from multiple contributors, giving preference to the most recent and widely confirmed information helping to keep the bike path inventory up-to-date and relevant for all users.

Also, registered users can record their individual trips, obtaining detailed statistics such as total distance, average speed, and travel duration. The system also integrates meteorological data from external services, enriching trip records with real-time weather conditions like temperature, wind speed, and precipitation.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Ultimately, the system aims to enhance cycling safety, promote community engagement, sport activities on the bike, and support sustainable mobility by providing up-to-date, user-verified information about the best available bike paths.

1.2.1 World Phenomena

WP1: Users ride bikes in paths.

WP2: Users choose specific paths to ride.

WP3: Weather conditions affects visibility and riding performance of riders.

WP4: Physical obstacles on the path may be encountered, which may affect users by continuing the riding.

WP5: External services provides information about the weather forecasting.

WP6: Road go under maintenance in certain periods.

WP7: Road gets closed for certain events.

WP8: Road gets modified.

WP9: Road gets permanently closed.

WP10: Road deteriorates over time.

1.2.2 Shared Phenomena

SP1: (If automatic mode enabled) The system collects the information from the registered user traversing paths.

SP2: (If automatic mode enabled) Registered user confirms path information.

SP3: (If automatic mode disabled) Registered User provides path information manually.

SP4: User search for paths.

SP5: Registered user publishes a new path.

SP6: Registered user updates a road status.

SP7: (If automatic mode enabled) Registered user starts riding.

SP8: (If automatic mode enabled) Registered user stops riding.

SP9: Registered user enables automatic mode.

SP10: Registered user disables automatic mode.

SP11: Registered user start recording a trip.

SP12: Registered user stops recording a trip

1.3 Definitions, Acronyms, Abbreviations

1.3.1 Definitions

- **Registered User**: A person who creates an account in the BBP system to access advanced features such as recording and publishing trips, adding or confirming path information, and contributing data either manually or automatically.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

- **Path**: A cycling route between a starting and finishing point recorded in the system. It consists of one or more streets or segments where bicycles can circulate under suitable conditions. Each path has associated information about its status, obstacles, and overall score. This score is computed based on the status of the path and on how effective the path is to let the user reach the destination from the given origin.
- **Trip**: An individual cycling activity performed by a registered user. It includes data such as total distance, average speed, duration, and, when available, weather information related to the ride.
- **External Service**: A data source outside the BBP system used to enrich trip information, particularly regarding weather conditions such as temperature, wind speed, and general climate.
- **Automatic Mode**: A data insertion mode in which BBP automatically collects information from the user's mobile device during the ride. It uses GPS, accelerometer, and gyroscope data to reconstruct the followed path and detect irregularities such as potholes, which the user must later confirm or correct.
- **Manual Mode**: A data insertion mode in which the user manually inputs information about a path, specifying streets, path conditions, and any relevant obstacles or issues observed during the ride.

1.3.2 Acronyms

- **BBP**: Best Bike Paths
- **SDK**: Software Development Kit
- **DBMS**: Database Management System
- **GPS**: Global Positioning System
- **WRS**: Weather Reporting System

1.3.3 Abbreviations

- **G**: Goal
- **WP**: World Phenomena
- **SP**: Shared Phenomena

1.4 Revision History

- 23/12/2025 - Version 1.0:
  + Added Section 1
  + Added Section 2
  + Added Section 3
  + Added Section 4
- 07/01/2026 - Version 2.0
  + Updated Domain Class Diagram: added fields to Trip Class regarding longitude and latitude
  + Updated order of execution of Login: now we check before for the verification and later for the credentials
  + Changed functional requirement phrasing: now the path information in automated mode is not saved "locally"

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

1.5 Reference Documents

The assignment for this document and all the information included refer to the following documentation:

- The specification for the 2025/26 Requirement Engineering and Design Project for the Software Engineering II course.
- The slides on the webeep page of the Software Engineering II course.

1.6 Document Structure

1. **Introduction**: a brief description of the project that follows from the analysis of the specification provided, it highlights the goals of the product and its scope.
2. **Overall Description**: a more detailed description of the product that comprehends also real world scenarios and analyses the system from a technical point of view.
3. **Specific Requirements**: a detailed list of all the requirements needed in order to satisfy the goals, and their relationship with domain assumptions
4. **Formal analysis using Alloy**: a formal description of a portion of real world phenomena analyzed through Alloy in order to highlight inconsistencies between assumptions and phenomena previously modelized.
5. **Effort spent**: the time spent by each member of the group to contribute to the realization of this document.
6. **References**: reference to documents or tools used for writing this document.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

2 Overall Description

2.1 Product perspective

2.1.1 Scenarios

User wants to register with an account to BBP

A user U wants to join the BBP community. U downloads the app from their favorite AppStore and opens it. U then goes to the registration page, enters their data (Username, Email, Password) and proceeds to the authentication phase. A message is sent to their email address and U must confirm their account. After that, they are ready to log in

Registered User wants to log in with their account to BBP

A registered user RU wants to log into their BBP account. RU opens the application on their smartphone and goes to the Login Page. Here, RU inserts their Username and their Password. BBP will hash the password and try to match the fingerprint with the one associated to RU account. If it is correct, the login process ends. If not, an error message appears and another attempt at login can be done.

Registered user enables automated mode

A registered user RU logs into the BBP application and enables the automated mode in the home page. The application prompts RU into authorizing access to the device's sensors (GPS, accelerometer, and gyroscope) as well as background execution to allow the system to start collecting data from their mobile device. The application then starts monitoring things like speed, vibrations, among other data to determine whether the user is riding a bike or not.

Registered user disables automated mode

A registered user RU disables automated mode. After this action, BBP no longer collects location or sensor data automatically. Future paths must be recorded manually if the user wants to add new ones. The user can re-enable automated mode at any time.

Registered user does a bike ride with automated mode

A registered user RU starts cycling with automated mode enabled. BBP detects the user's speed and movement, confirming that the user is biking. During the ride, the system collects GPS coordinates and motion sensor data to reconstruct the path and identify possible obstacles. After the sensors detect that RU stopped cycling, BBP starts processing the just collected data and notifies RU (with a pop-up notification on their phone) that the path has been recorded and that needs revision to be published. RU can decide either to review the entry, or to ignore it and delete it.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Registered user wants to publish a path recorded in automated mode

After their cycling activity, a registered user RU wants to publish the recently recorded path, and make it available for every other user. In doing this, they either press the pop-up notification or open the application by pressing the icon. BBP shows them a summary of the path, including detected issues. RU reviews the information, and changes the incorrect or problematic parts (if any). Finally, RU can publish the entry to the application, ready to be used by BBP.

Registered user publishes a bike path in manual mode

A registered user RU accesses the manual entry section in BBP to add a new bike path. RU specifies the streets that compose the path and assigns a status to each road that composes the path. The user adds details like the presence of obstacles and surface type. Once the information is complete and reviewed, RU publishes the path, allowing it to become visible to all the user base.

Registered user records a trip

A registered user RU begins recording a new cycling session using the BBP application. If the access of the sensors (GPS, accelerometer, and gyroscope) and background execution has not been granted yet, the application will prompt the user to allow access. BBP records the trip details through the device's sensors, including: start and end timestamps, total distance covered, and average speed. Upon completion, RU can save the trip to their personal log and view statistics. Asynchronously, once the trip is finished, BBP will use a external service that provides weather data to enrich the trip's data with information such as: Temperature, wind speed, and precipitation.

User searches for paths

A user U, whether registered or not, specifies an origin O and destination D. BBP calculates, using the road stored in its database, all the possible bike paths between O and D. If multiple options are available, BBP displays them ranked by their path score, which depends on path status (calculated as a mean of all the statuses of the single roads in the path) and effectiveness. U can visualize the suggested routes on the map and explore details about obstacles and path condition.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

2.1.2 Domain Class Diagrams

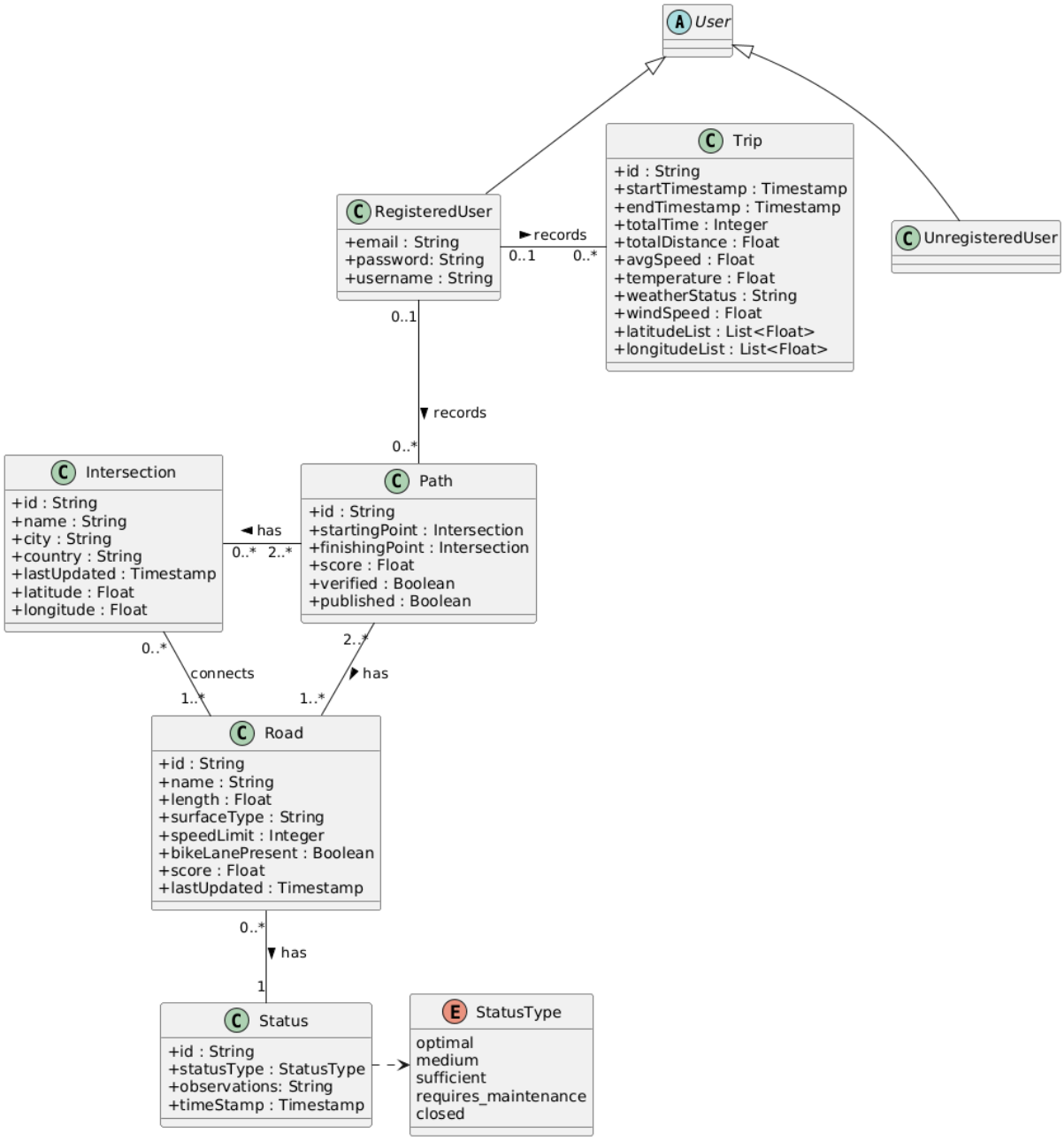


Figure 1: Domain class diagram for BBP.

This Domain Class Diagram [2] represents the entities involved in the domain of our system and the relationships between them. Some important points to note:

- The system distinguishes between registered and unregistered users, with registered users capable of recording trips, contributing data, and making information publishable, while unregistered users can only access information but not contribute or personalize their data.
- Registered users can record trips, which include detailed statistics like total time, total distance, average speed, and meteorological data (weather, temperature, wind speed) when available, supporting richer activity logging.

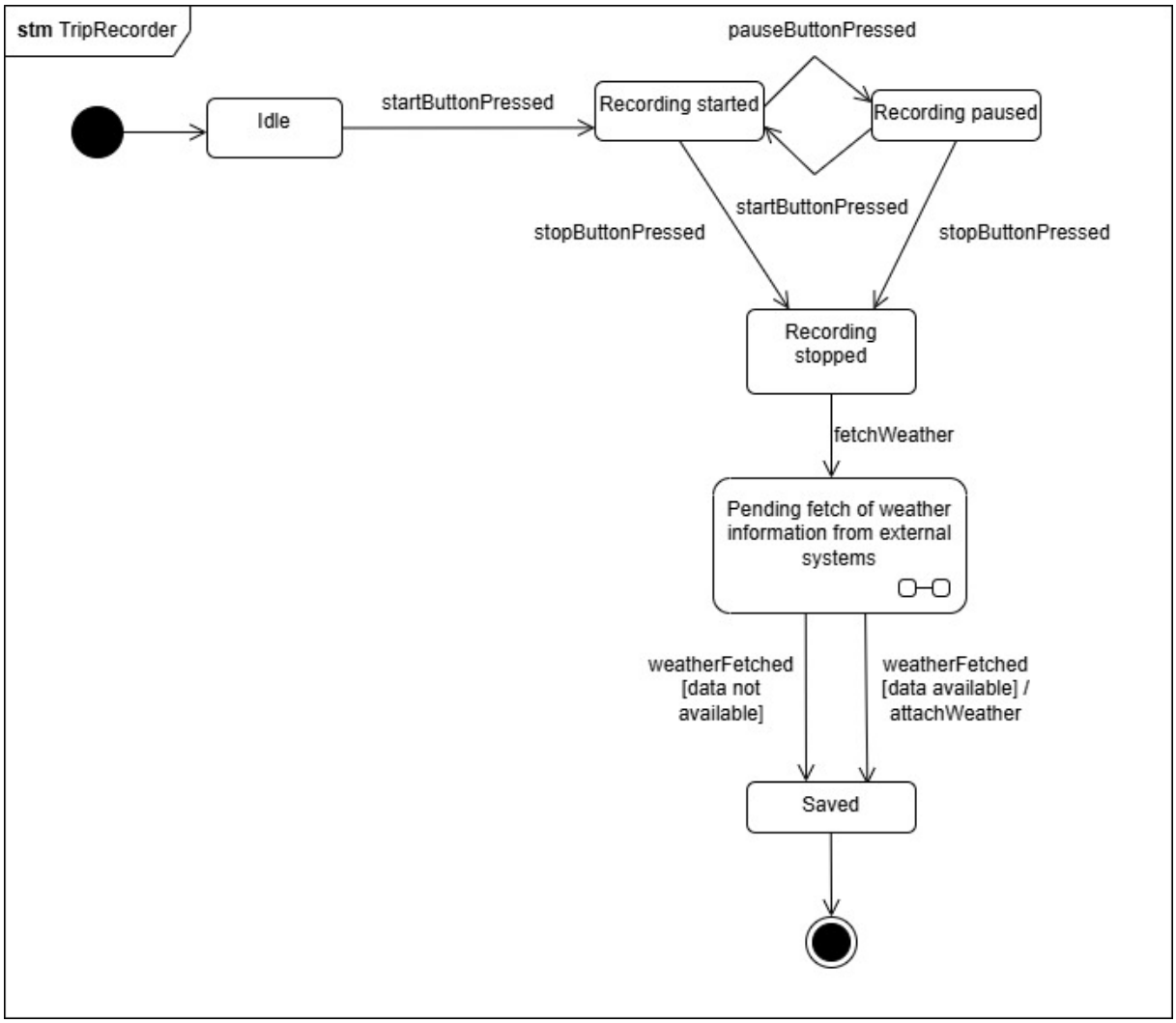
Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

- Paths connect intersections (nodes), utilizing roads as their links, mirroring the real-world and supporting route computation for origin-destination queries. Each path can be marked as verified, published, and scored, facilitating ranking and quality assessment based on both user input and system-calculated factors. Roads and their status types (e.g., optimal, requires maintenance, etc.) allow precise condition reporting, which is crucial for route planning and safety.
- The system allows both manual and automated data entry for path and obstacle information, supporting flexibility for user input and leveraging sensor data for finer granularity. Automated mode requires post-capture user verification, addressing the problem of false positives (e.g., erroneous pothole detections), ensuring data quality before publication.
- The system supports community-driven data integration, where multiple user contributions about the same path or road are aggregated, prioritizing information based on freshness and majority consensus for better accuracy and reliability.
- Both registered and unregistered users can visualize computed paths based on road status and path effectiveness , helping users choose optimal routes.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

2.1.3 State Machine Diagrams

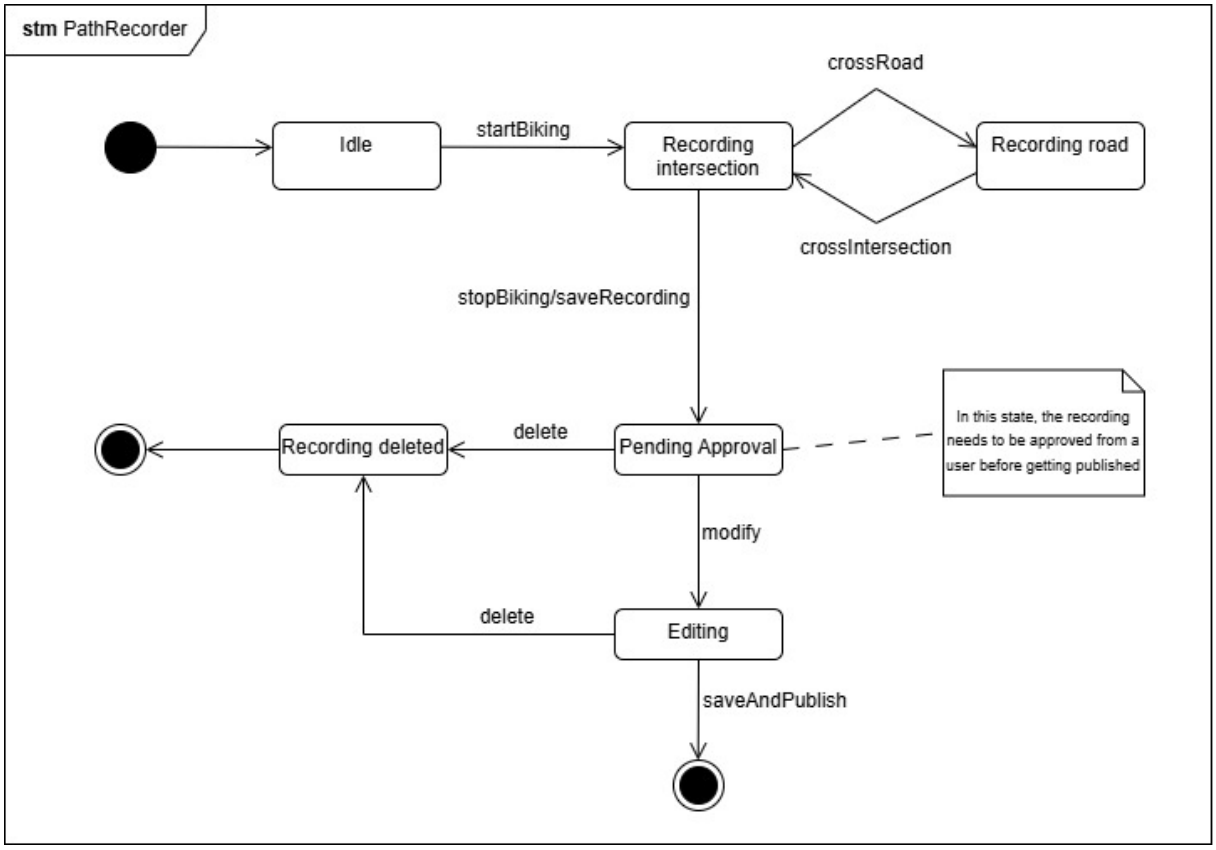
[3] The registered user records a Trip



This state machine describes the different states in which the TripRecorder can be found. At first, the machine is in idle state, waiting for the user to decide if they want to start a recording.
After pressing the start button, the application starts recording, acquiring information on the streets and intersections crossed by the biker, their average speed, max speed, time passed and distance traveled. The user can decide, at any time, to pause and resume the recording while cycling.
At the end of the entire trip, the stop button is pressed and the data is processed.
During data processing, the application asks an external weather reporter system additional data, to enrich trip details. If this information is present, they are attached to the trip recording.
Regardless of the outcome, the trip is saved and now ready to be reviewed.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

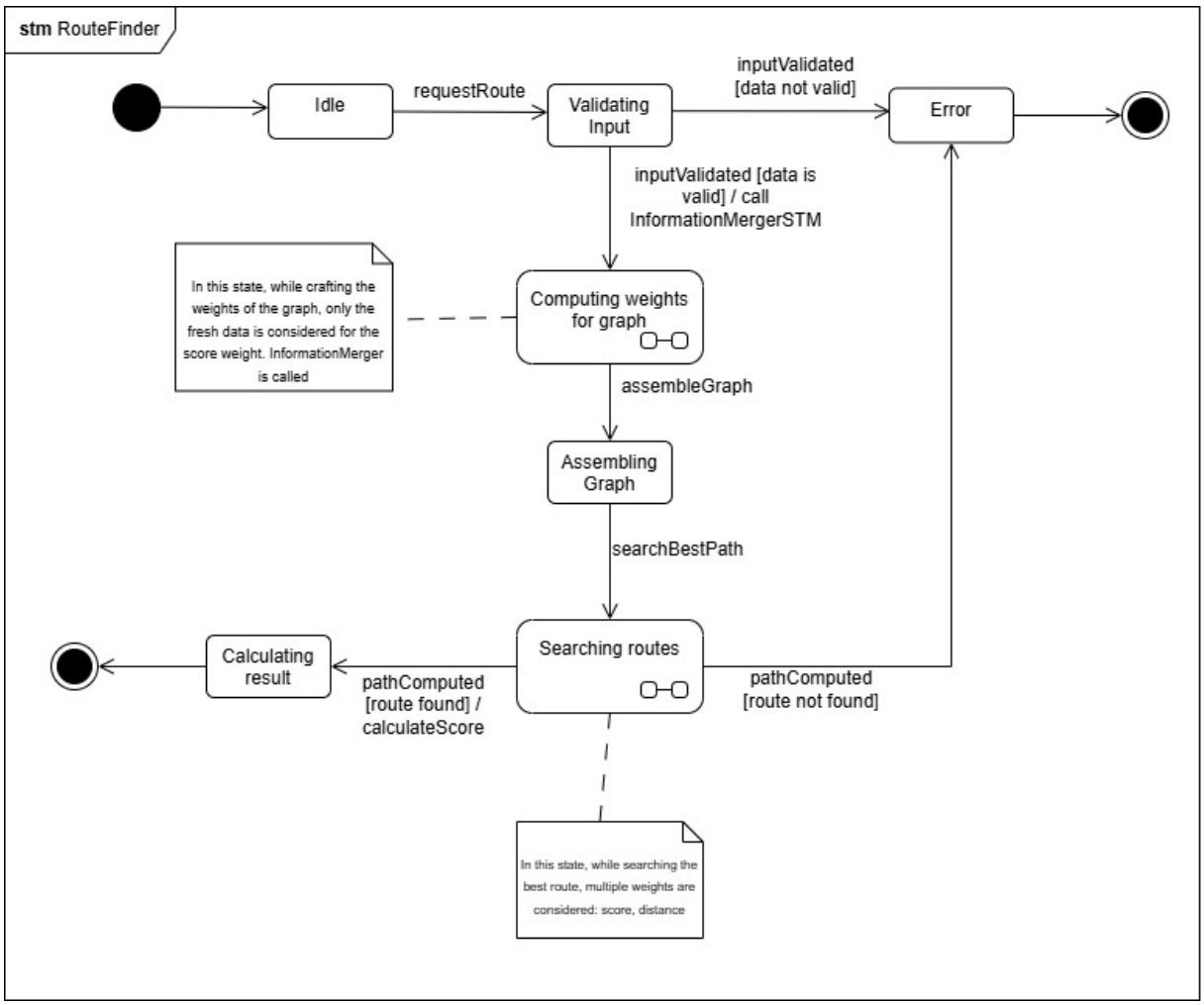
The registered user records a Path



This state machine describes the different states in which the PathRecorder can be found. Please note that this diagram is showing the **Automated Mode** only. The **Manual Mode** is very similar, and journaling is delegated to the user.
After selecting the automated mode in the home page, the machine goes in idle state, waiting for the user to start biking.
When the external sensors (GPS, gyroscope, accelerometer) detect a possible cycling motion, the application starts recording the cycling activity. At this point, roads and intersections information are acquired using the same sensors (especially estimating road conditions and position into the map) until the biker stops cycling.
All the collected data is then saved and ready to be reviewed. Considering the possibility of having false positives, the path must be reviewed by the user before it reaches the published state. In any moment of this process, the path can be discarded and never published.
At the end of the editing phase, the path is saved and published, now available to be used by the application to find paths for other users.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

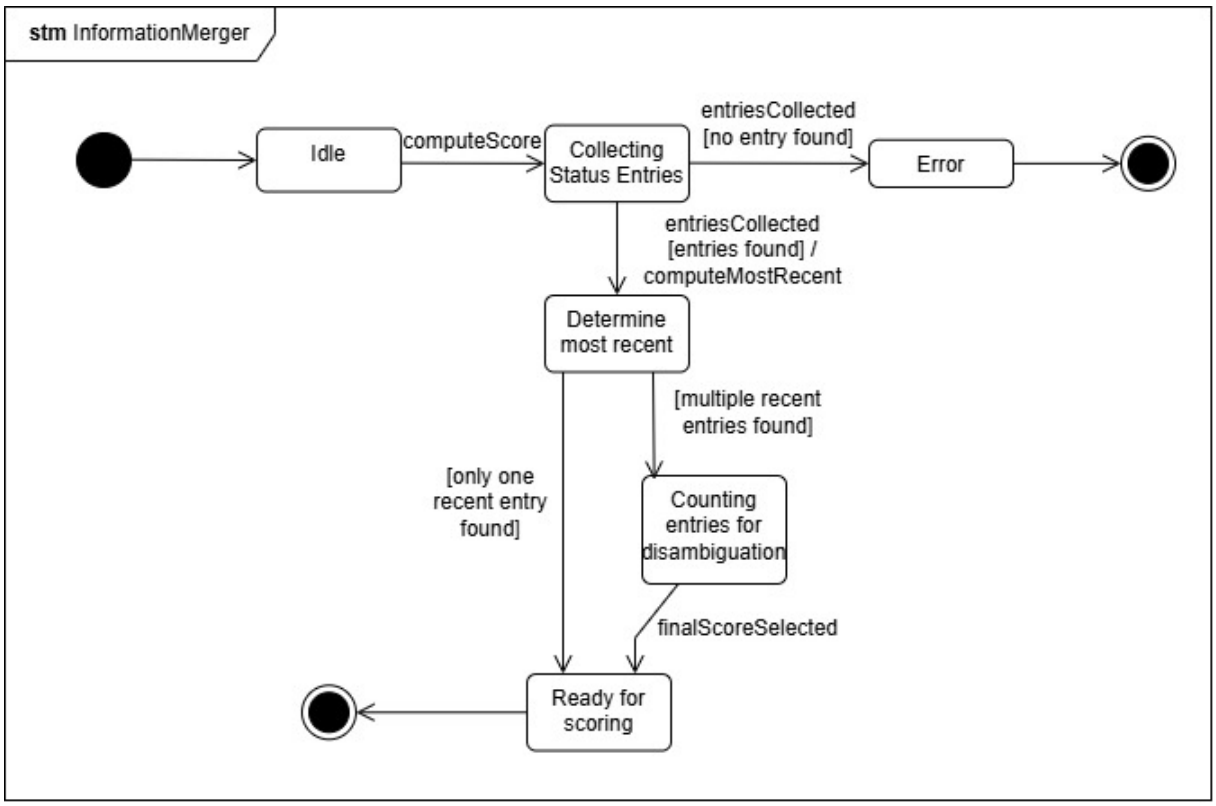
User searches for Paths



This state machine describes the different states in which the RouteFinder can be found. This machine is used to calculate the possible routes between a starting point and an ending point selected by the user.
After inserting the start and ending positions, a request to the machine is sent and the input validated. An error occurs if the positions do not exist inside BBP.
The chosen model to represent this problem is a graph, with intersections as nodes and roads as weighted arcs. Please note that, if the starting point is not an intersection but positioned inside the road, an additional parameter is multiplied to the weights to express the real position inside the graph. These weights need to be retrieved from BBP database. In particular, the roads "conditions" could be ambiguous due to multiple entries added at the same time. To solve this issue, the condition weight is calculated by another machine (see InformationMergerSTM below) to evaluate the real current condition of the road.
The graph is then built and routes found by using classical route searching algorithms in graphs. For each route, a score is calculated to be correctly displayed in the mobile application.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

The road score is calculated



This state machine describes the different state in which the InformationMerger can be found. This machine is used to merge and present the current and most fresh information to the user.
When it is asked to compute a specific road condition, the machine fetches the status entries from the application database regarding that specific road and uses the "timestamp" attribute to determine the most recent entry.
To consider also the possibility of multiple entries with the same timestamp, if the machine finds this "ambiguity", it also performs a "disambiguation" operation: it counts, for every different label of the road condition, the entries with the same timestamp and returns only the one with the biggest cardinality. If even in this case, it's present an ambiguity, it returns the worst label of the group (this guarantees the safety of the biker, avoiding any possible injury).

2.2 Product functions

Sign Up

Users can create a new account by providing the necessary personal information to register with the Best Bike Paths (BBP) system. The required personal information includes:

1. Username
2. Email
3. Password

After submitting the registration form, a confirmation email will be sent to the user's email address. Once the user

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

confirms their account, they will be considered successfully registered and will be redirected to the login page to perform their first login.

Login

The BBP application allows users to sign in using an email address and a password. After inserting the credentials, BBP will check the correctness of the information (f.e. that an email address is associated to an account, or that the account has been successfully verified) and will redirect the user to the home page if that is the case. If an error during this step has occurred, BBP will notify the user about it.

Record a trip

Registered users can record their cycling activities using the BBP application. When a user starts trip recording, the system collects data from GPS, accelerometers, and gyroscopes to accurately track the route and motion. Upon trip completion, BBP processes the collected data to calculate various statistics, including total distance covered, average speed, and total time spent. The recorded trip is then saved to the user's account and can be viewed through the application interface.

Enrich a trip's data

Once a trip is uploaded to BBP, an asynchronous process begins to enrich the trip data with meteorological information such as weather conditions, temperature, and wind speed. This data is retrieved from an external service and then integrated into the existing trip record. After the trip is updated, users can view the additional meteorological details on the trip detail interface.

Insert and publish a bike path

The BBP application offers two modes for collecting raw data, depending on the user's preference.

1. **Automated Mode**
   In the automated mode, the BBP application leverages all the built-in sensors of the user's device to automatically gather the necessary raw data for reconstructing the complete path. Specifically, the GPS sensor is used to track the user's journey, while motion sensors such as the gyroscope and accelerometer help assess the difficulty of the path at each point—enabling the detection of potholes or other irregularities affecting the ride quality. To improve the reliability of the collected data, users are encouraged to review and confirm the system's detections once the journey is completed.
2. **Manual Mode**
   The manual mode is simpler and entirely relies on the user's honesty and accuracy when entering information. In this case, the user must manually specify the names of the streets included in the path and indicate their respective conditions. If the condition of a path is not "Optimal", the user can also add details about the problems encountered.

Browse bike paths

Any user can explore available bike paths by specifying origin and destination points. The system displays the routes on a map, presenting relevant information such as path conditions, obstacles, and maintenance status to

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

assist in route selection. When multiple paths are available, BBP visualizes them and assigns a score to each, reflecting the quality of the path based on its effectiveness and current condition. This scoring helps users evaluate and choose the most suitable route for their needs.

Score a path

The BBP system collects information from multiple paths and visualizes it through a scoring mechanism. The score is primarily based on the condition of the path and its effectiveness in enabling the user to reach the desired destination from a given origin. The score is given by the user using tags.

Merging of similar paths and updating statuses

When multiple users provide information about the same bike path, BBP merges the data by evaluating the recency of each input and the number of confirming reports. This merging process automatically updates the path's status to reflect the most accurate, consensus-driven information. Users may tag paths with statuses such as "Optimal" or "Requiring maintenance." During data consolidation, BBP prioritizes the tag from the most recent input to ensure the path status remains up to date.

2.3 Users characteristics

Any user — whether an experienced rider or completely new to biking — can join the platform through the application. Each path shared by a user remains anonymous and is not linked to its creator. As a result, all related paths are automatically collected and anonymously made available to other users. The user is able to insert information, scores of hardness, and any kind of information to any path taken. Furthermore, the user can even relies on the platform to automatically generate the information, however, a final check to the information must be provided. Every user profile and information are private and remain hidden from other users.

2.3.1 Any user (non-registered and registered users)

All users, whether registered or not, can explore the information provided by Best Bike Paths (BBP). They can use the map interface to enter an origin and a destination, asking BBP to show possible bike routes between those points. When several options exist, the system ranks them by a score based on path condition, safety, and route effectiveness.
Users can view each path's detailed information, including maintenance status, surface quality, and possible obstacles like potholes. However, unregistered users cannot record trips or add new information, but still access BBP's main navigation and route-finding tools supported by community data.

2.3.2 Registered users

Registered users gain access to extended features supporting individualized interaction and community contribution. They may record personal cycling activities, which are stored within their user profiles and supplemented by metrics such as total distance traveled, average speed, and detailed trip analytics. BBP may augment recorded trips with meteorological data, including weather conditions and temperature, sourced from external services.
Furthermore, registered users can contribute path information via two mechanisms: manual and automated. Manual mode allows users to annotate routes with names and assign path statuses. Automated mode enables BBP to collect and analyze path data in real time using mobile device sensors, subsequently allowing users to confirm or revise detected obstacles prior to publication. Validated contributions may be made publishable, with BBP

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

consolidating multiple reports to reflect the most current and consistent community knowledge. These functionalities ensure that registered users can both improve and benefit from the evolving quality of information within the platform.

2.4 Assumptions, dependencies and constraints

2.4.1 Regulatory policies

The application is designed to enhance the overall user experience by providing an intuitive and efficient interface for data collection and sharing paths. Any user, regardless of their level of experience, can contribute to the platform by recording and publishing biking paths through the application. All related paths are automatically gathered and processed without any association with their original creator, ensuring that personal information remains strictly confidential. This approach guaranties full compliance with the General Data Protection Regulation (GDPR) [6] and establishes a secure environment in which users can freely share useful data while maintaining anonymity. Furthermore, personal data are used exclusively for authentication and registration purposes — specifically for log-in via email, account setup, and authorization to access device sensors or third-party services. No personal information is used for any other purpose.

2.4.2 Domain Assumptions

- **D1**: Each user rides a bicycle while the application runs in background in case of automated mode.
- **D2**: The internet connection remains relatively stable throughout the session.
- **D3**: Users are willing to review and refine the automatically collected information gathered by the device in the automated-mode.
- **D4**: Users are expected to provide accurate and non-misleading information when creating new paths.
- **D5**: Users do not use the application or the platform for purposes unrelated to biking activities or the exploration of new routes as a recreational activity.
- **D6**: Third-parties services provide reliable information to the application.
- **D7**: Hardware devices must provide reliable sensor data to the system to let it acquire the right performance data.
- **D8**: The date related to the new inserted path is correct.
- **D9**: The app is used where bike track already exists.
- **D10**: The biking paths have little to no cars presence.
- **D11**: The biking paths have speed limits compatible with the average speed of a bike.
- **D12**: Users will stop cycling when they want to end the recording of the path.
- **D13**: Users will give non-misleading information about their identity.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

Here is shown a mockup of the mobile application UI. The images outline the user interface of the most important screens from the point of view of a generic user, both registered and unregistered. The idea is to implement a easy-to-use mobile application interface, with the possibility to navigate through the pages using a navigation bar and buttons. Further details are provided in the Design Document of the application.

User Interfaces for Registered and Unregistered Users

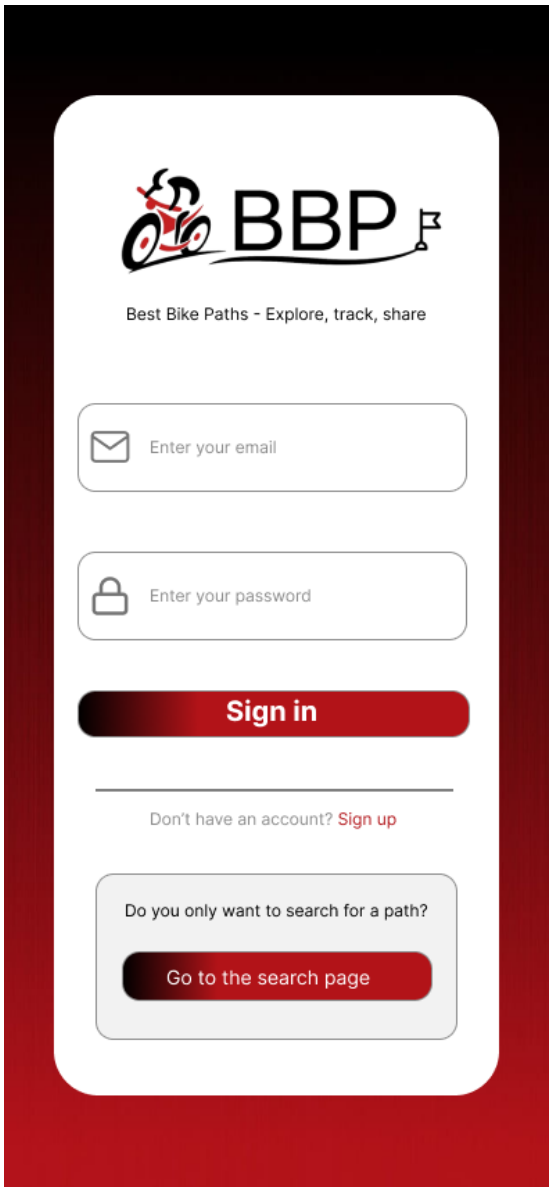


Figure 2: Sign in page of BBP.

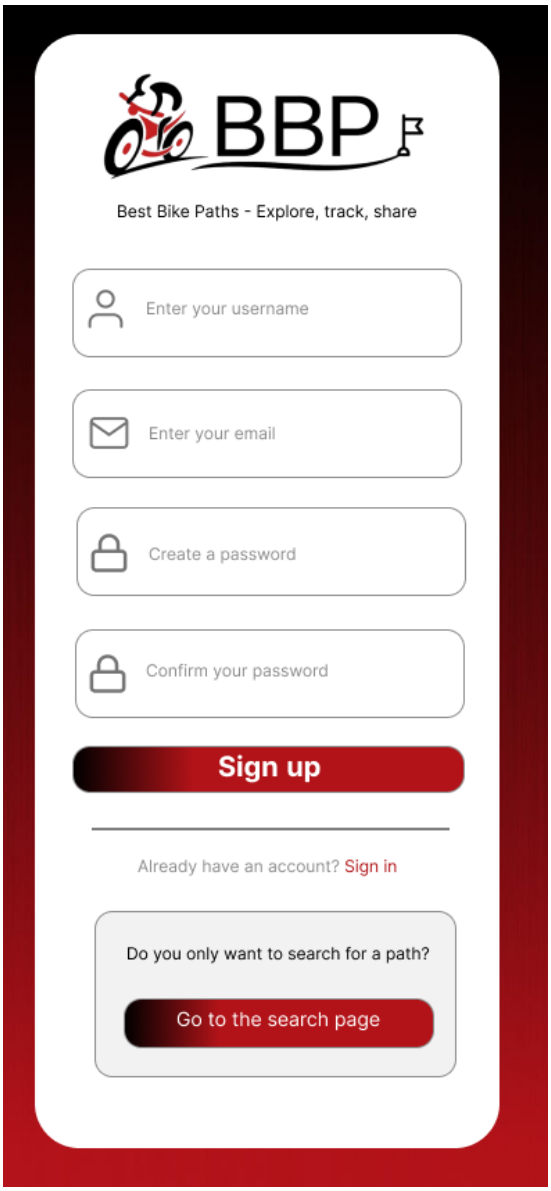
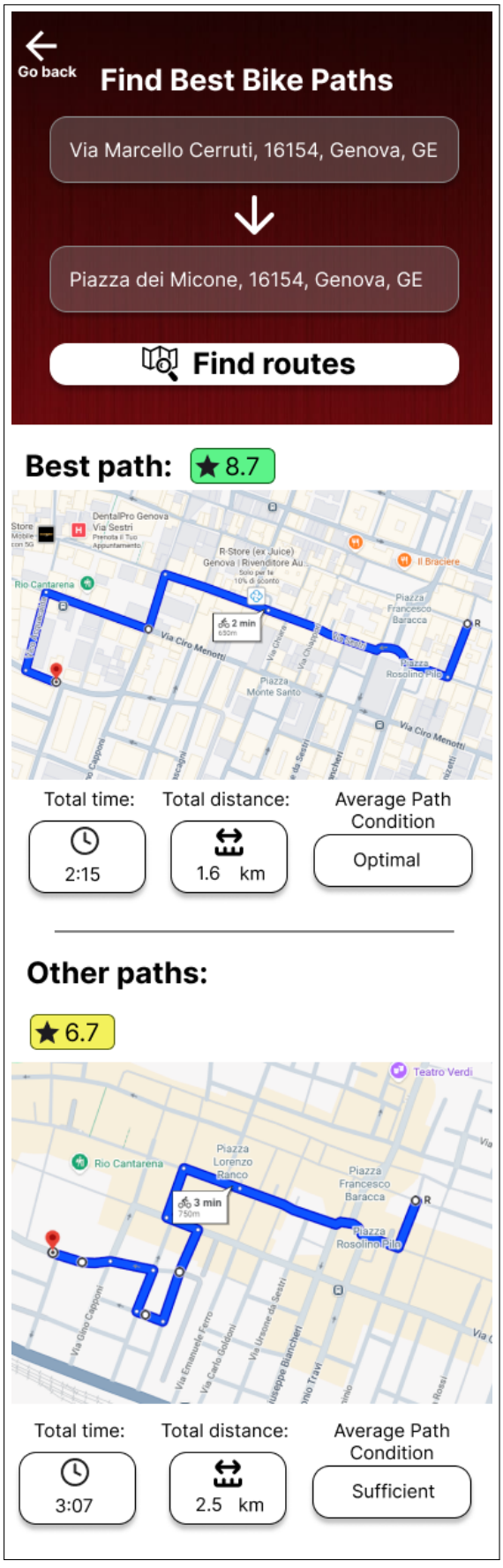


Figure 3: Sign up page of BBP.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved



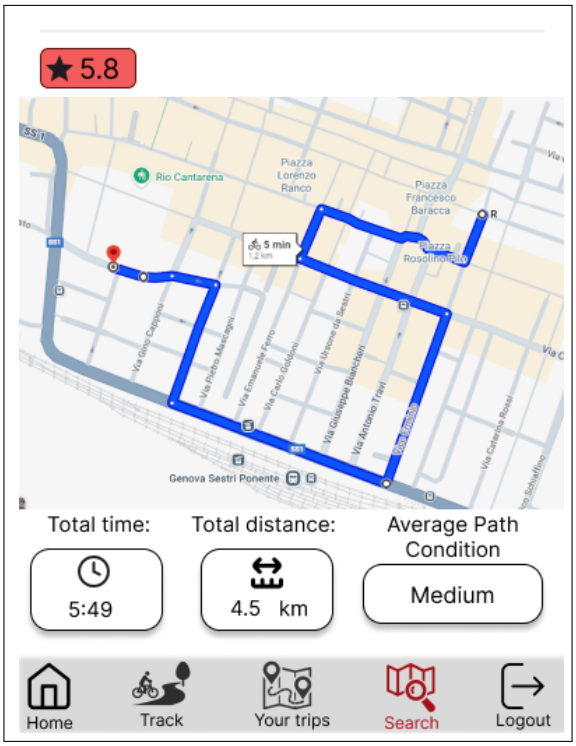


Figure 4: Possible screen visualization after a search in BBP. Note: If a User is not registered, the navigation bar is not present

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

User Interfaces for Registered Users only

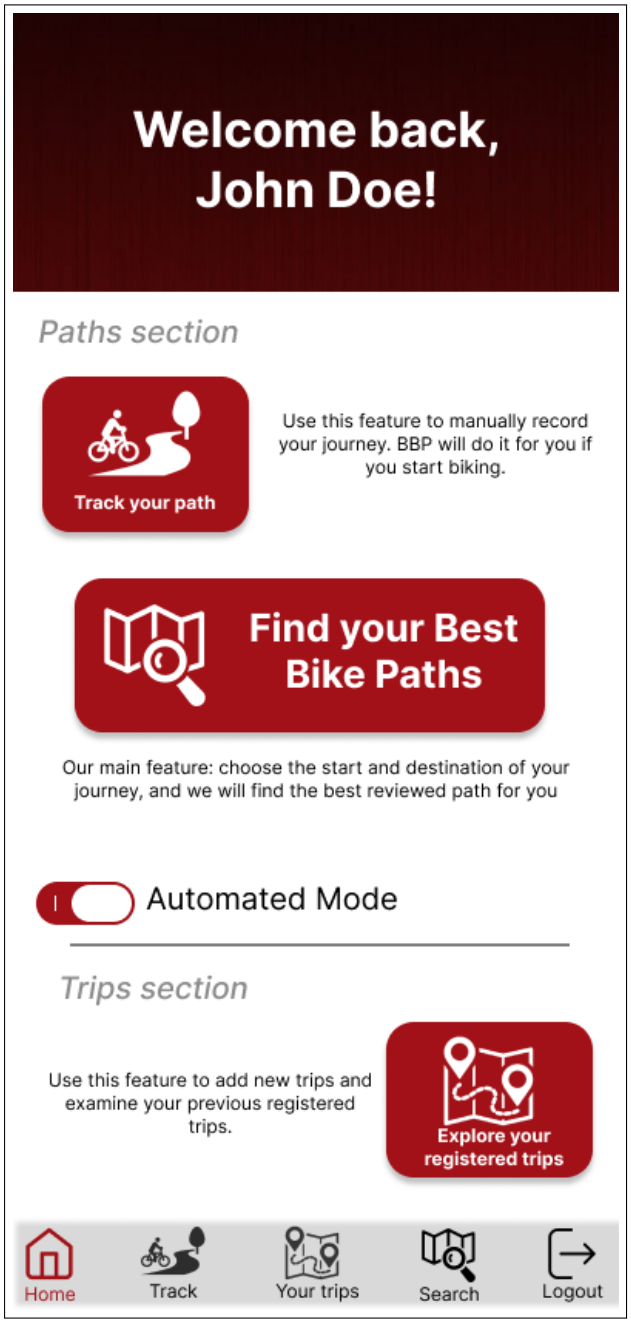


Figure 5: Home page of BBP.

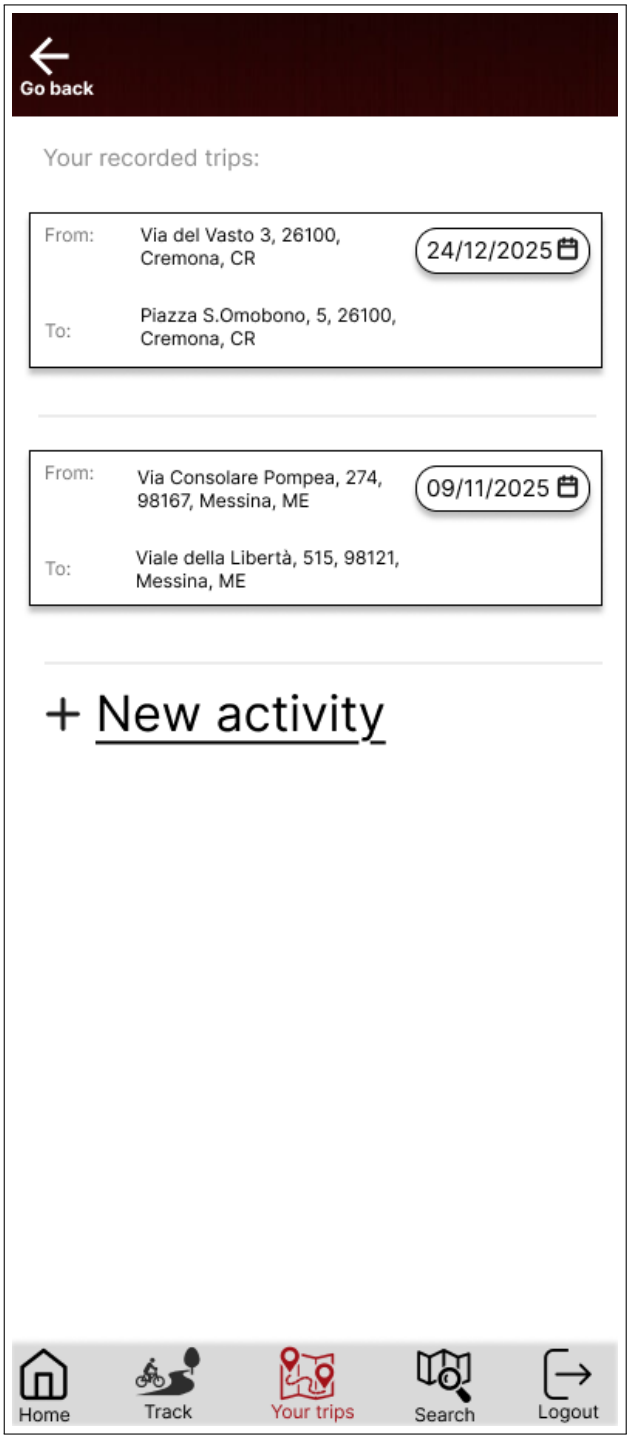


Figure 6: List of trips registered (not published paths).

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

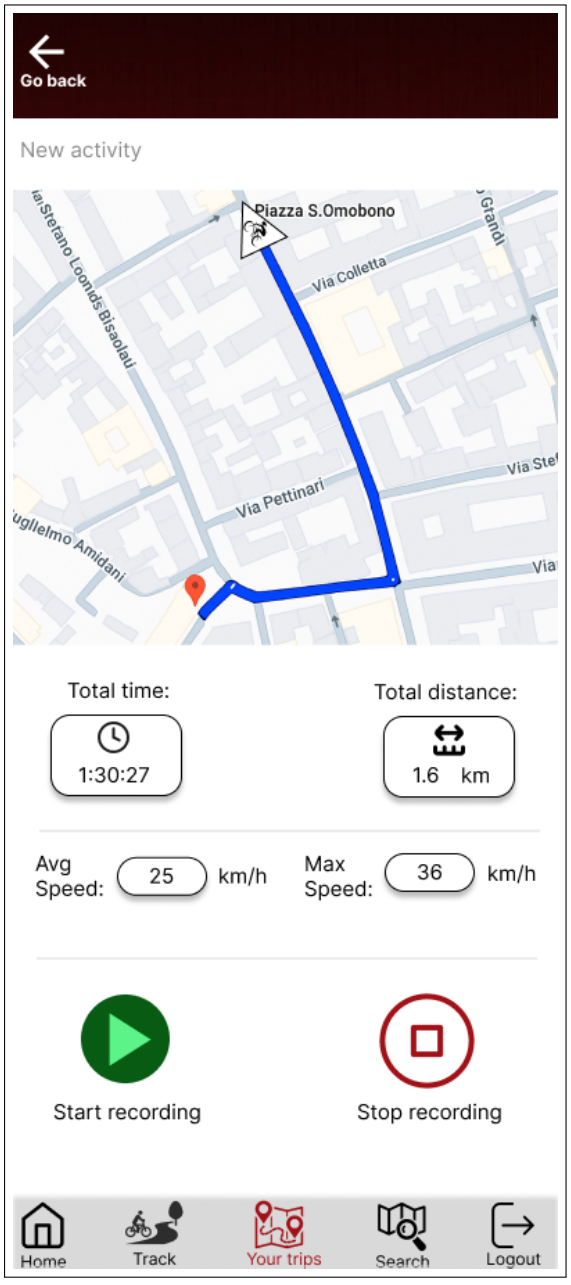


Figure 7: Recording a new activity using BBP.

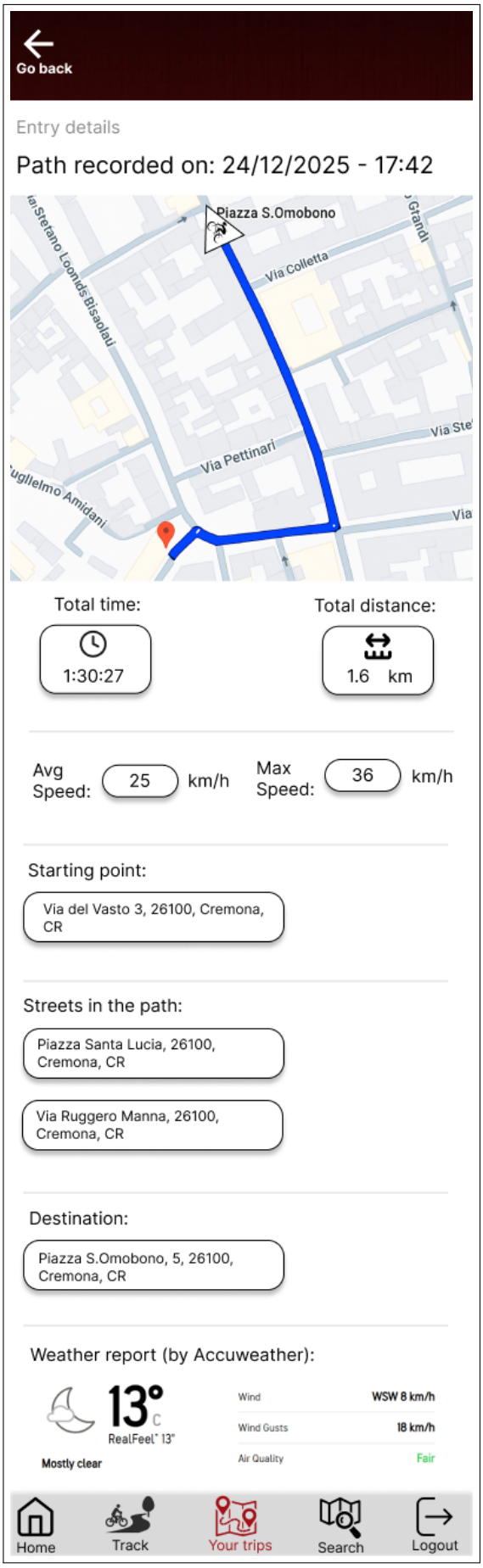


Figure 8: Reviewing your recorded activities using BBP.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

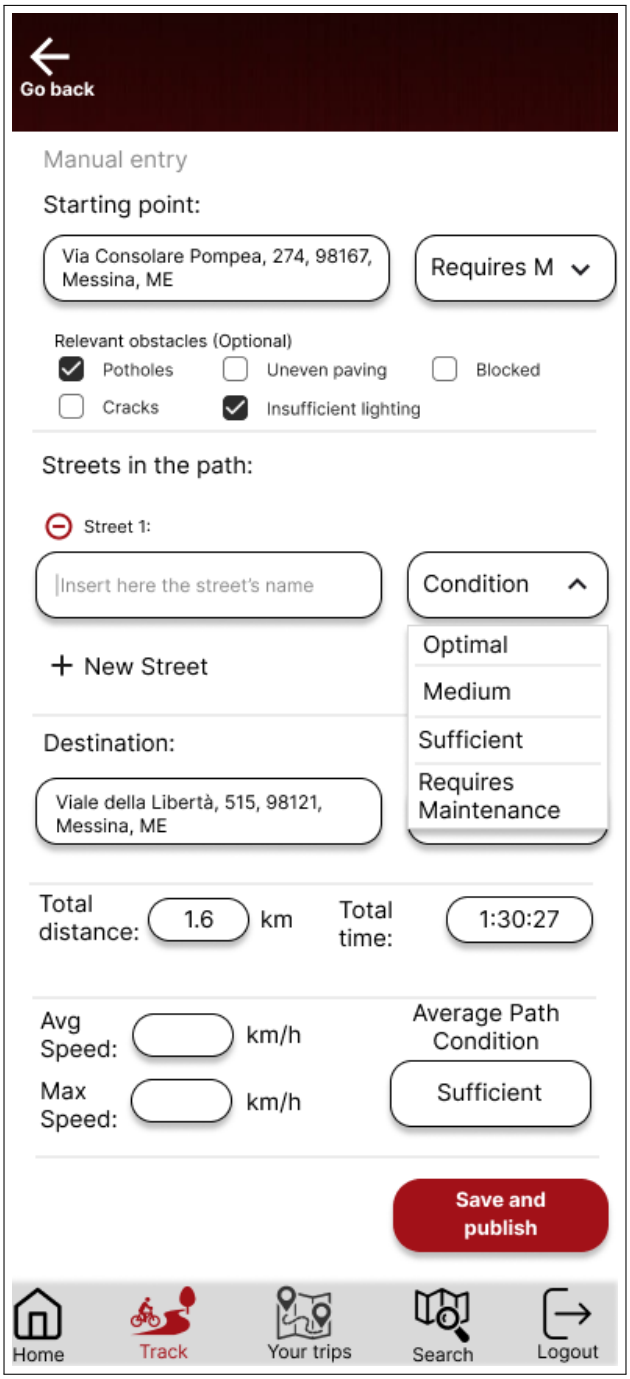


Figure 9: Page used to insert tracked paths to be published and made available to every user.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

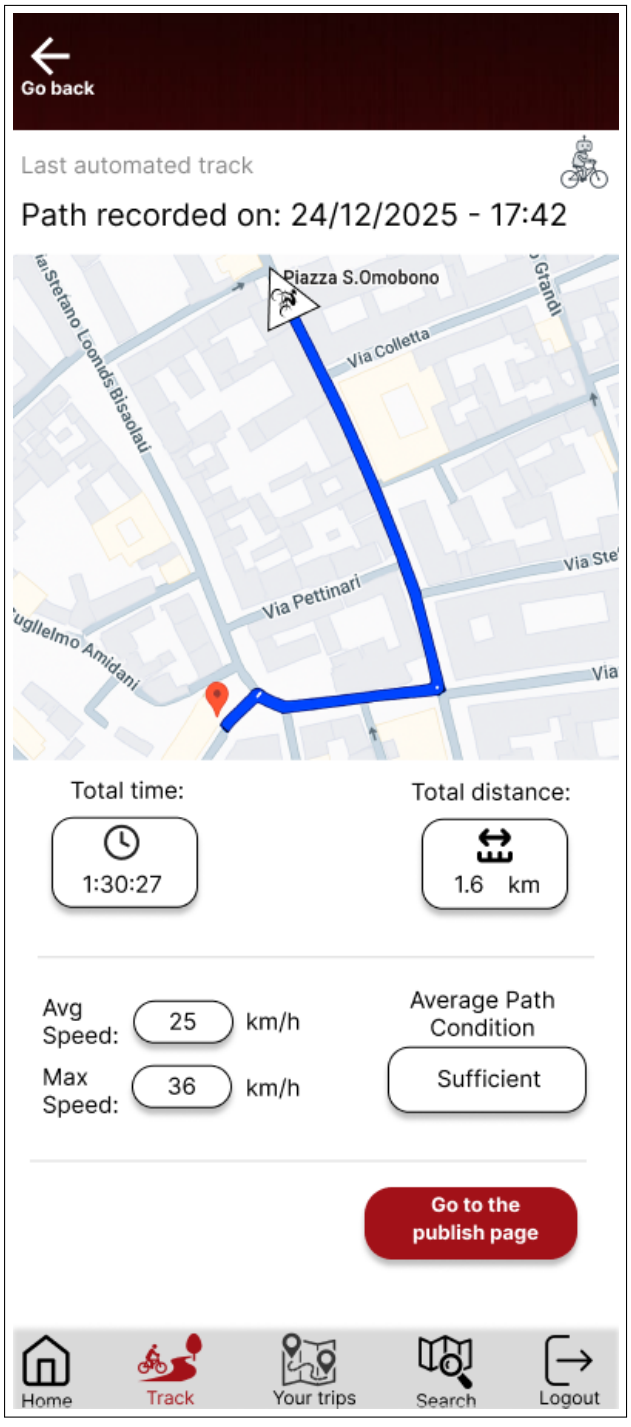


Figure 10: After doing an automated tracking of a path, this is the first screen the user sees when booting up the app.

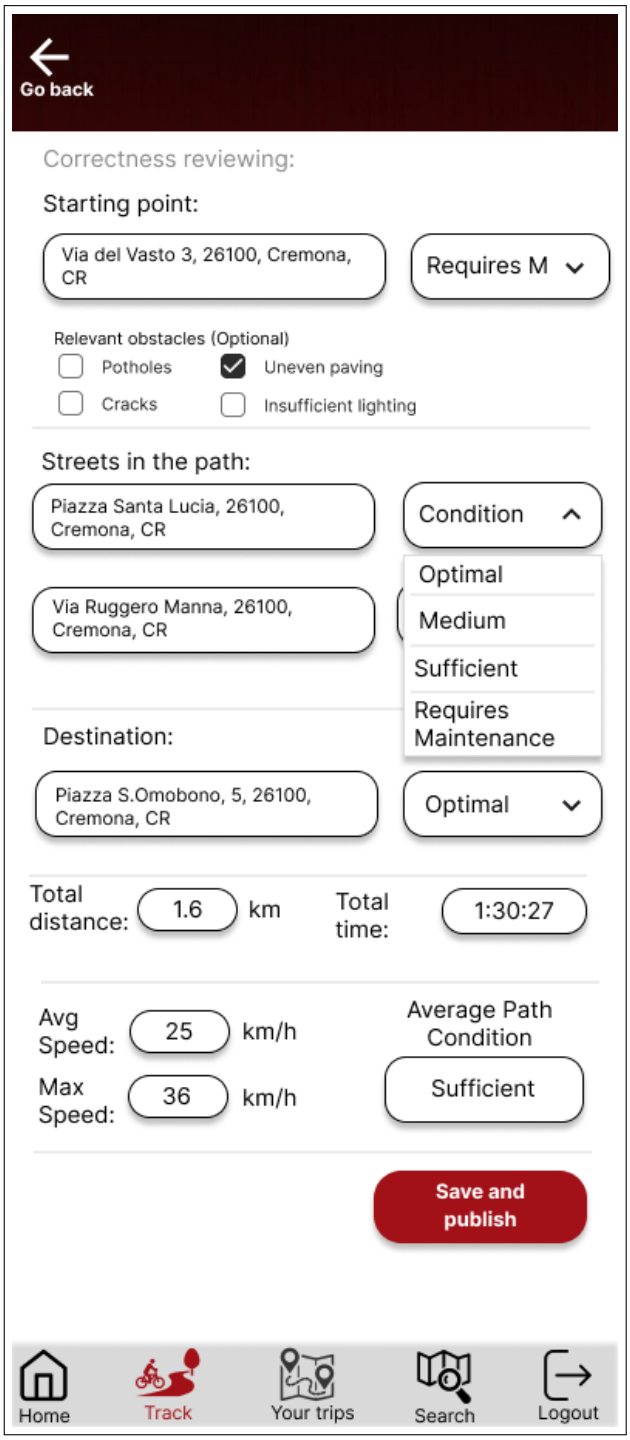


Figure 11: Reviewing the automated tracking paths before publishing them.

3.1.2 Hardware Interfaces

Our platform is a mobile application, requiring a portable device, such as a smartphone. Additional sensors are mandatory (either inside the device or connectable to be easily read by it):

- **GPS**: used to gather reliable information regarding the biker's position in the map. It is needed to also

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

accurately reconstruct the path performed by the user.

- **Gyroscope**: used to gather reliable information regarding the possible obstacles in the path (e.g. potholes).
- **Accelerometer**: used to gather reliable information regarding the possible obstacles in the path, the user's speed during the activity (maximum and average speed) and detect when the biker is starting or ending their activity (in automated mode).

A stable Internet connection is also required throughout the entirety of the recording, in particular during the saving of the recordings and the queries to search the best possible paths (both of these features require communication with the central server).

3.1.3 Software Interfaces

Email and Weather Report Systems

In order to enrich the user experience, an external email delivery system [1] is used to send confirmation emails. Furthermore, trip data can be enriched by weather report systems [11]. To achieve this, the system relies on REST APIs as interfaces to the explained services and components.

Map rendering

To make the user interface more accessible, a mapping SDK [7] is used by BBP to show on-screen a map of the current location and the possible paths. The SDK provides APIs to draw tiles, overlays, pointers.

DBMS

In order to store the entirety of the information that makes BPP work, a database and a management system is needed. Every DBMS provides APIs to perform actions to the data.

3.1.4 Communication Interfaces

Our system consists of two main parts: the mobile application and the backend system, that also communicate with several others components.

Communication between mobile application and system

The mobile application must communicate with the backend system using the Internet protocols. This means that each interaction, in particular data transmission, must be secure and encrypted using REST APIs with HTTPS. The purpose of this connection is in regard of the entirety of the user authentication process, the upload and the download of trip and path information.

Communication between mobile application and on-device sensors

The mobile application must communicate with on-device sensors to retrieve data to be sent to the server, here stored, processed and made available to others. This communication happens through specific APIs for each and every sensor.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Communication between server and email provider

The server must communicate with an email delivery system to send profile confirmation emails. For this purpose, REST APIs with HTTPS are still widely used. It is to note that the mobile application never communicates with the email provider system, all of the actions go through the server.

Communication between server and weather provider

The server must communicate with an external weather providing system to send queries about current weather reports and receive data to attach to trip information. For this purpose, REST APIs with HTTPS are still widely used. It is to note that the mobile application never communicates with the weather report system, all of the actions go through the server.

Communication between server and database

The server must communicate with a database system, to store and retrieve data about users, paths and trips. For this purpose, SQL queries are the basis of our communication.

3.2 Functional Requirements

Sign up

- R1: The system must allow users to register using a username, an email and a password.
- R2: The system must verify that credentials are unique.
- R3: The system must send a verification email to the user.
- R4: The system must let people verify their email after receiving the verification request.
- R5: The system must activate accounts only if verified.

Login and Logout

- R6: The system must allow registered users to log in using email and password.
- R7: The system must verify the correctness of the credentials.
- R8: The system must allow registered users to log in only if the credentials are correct.
- R9: The system must allow registered users, that logged-in, to log out.
- R10: The system must delete user's session.
- R11: The system must allow only registered users to record trips and submit new bike path information.
- R12: The system must allow every user to access the system to browse paths.

Permissions

- R13: The system must ask registered users to give permissions for the sensors.
- R14: The system must read the sensors data only if the permissions are given by the registered user.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Trip Management

- R15: The system must allow registered users to record their bike trips.
- R16: The system must use sensors to track the biker's trip and their performance metrics.
- R17: The system must allow the registered user to save the trip recorded.
- R18: The system must check the availability of the Weather Reporting Service (WRS) before requesting data.
- R19: The system must ask the WRS for weather information regarding the trip considered.
- R20: The system must attach weather information to the trip entry, if available.
- R21: The system must show registered users a list of their recorded trips.
- R22: The system must allow registered users to see details of a specified recorded trip.

Bike Path Management - Manual mode

- R23: The system must allow registered users to manually insert path information using a form.
- R24: The system must require the user to specify roads and intersections in the path.
- R25: The system must allow registered users to specify the condition of each road in the path.
- R26: The system must allow registered users to report additional issues (f.e. potholes).
- R27: The system must save the path information in the central database.

Bike Path Management - Automated mode

- R28: The system must let registered users enable automated mode.
- R29: The system must let registered users disable automated mode.
- R30: The system must let registered users add path information via automated mode.
- R31: The system must record data using sensors only if automated mode is enabled.
- R32: The system must recognize if the registered user is biking.
- R33: The system must start recording the path only if the system recognizes that the registered user is biking.
- R34: The system must use the sensors' data to record the path.
- R35: The system must record intersections and road crossed by the registered user.
- R36: The system must recognize obstacles in the path (f.e. pothole) using sensors' data.
- R37: The system must recognize if the registered user stopped biking.
- R38: The system must stop recording the path if the system recognizes that the registered user stopped biking.
- R39: The system must save the recorded bike path information in "to be reviewed" state after an automated mode recording
- R40: The system must notify the user that a path has been recorded this way.
- R41: The system must require registered users to review and edit information collected via automated mode before publishing.
- R42: The system must allow registered users to discard the information collected via automated mode.
- R43: The system must allow registered users to save and publish the information recorded via automated mode.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Finding the Best Path

- R44: The system must allow every user, regardless of their registration status, to browse for paths.
- R45: The system must allow the user to fill up a form regarding an origin and destination point.
- R46: The system must compute the best path(s) considering the specified points.
- R47: The system must merge information from multiple users regarding a road, considering freshness of information and number of confirming users.
- R48: The system must compute an average status for the specified path.
- R49: The system must compute a score for the specified path, considering their status and the easy reachability.
- R50: The system must show the user all the possible best paths in descendant score order, with their scores and metrics.
- R51: The system must show the user a map of the best path(s).

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

3.2.1 Use Case Diagram [5]

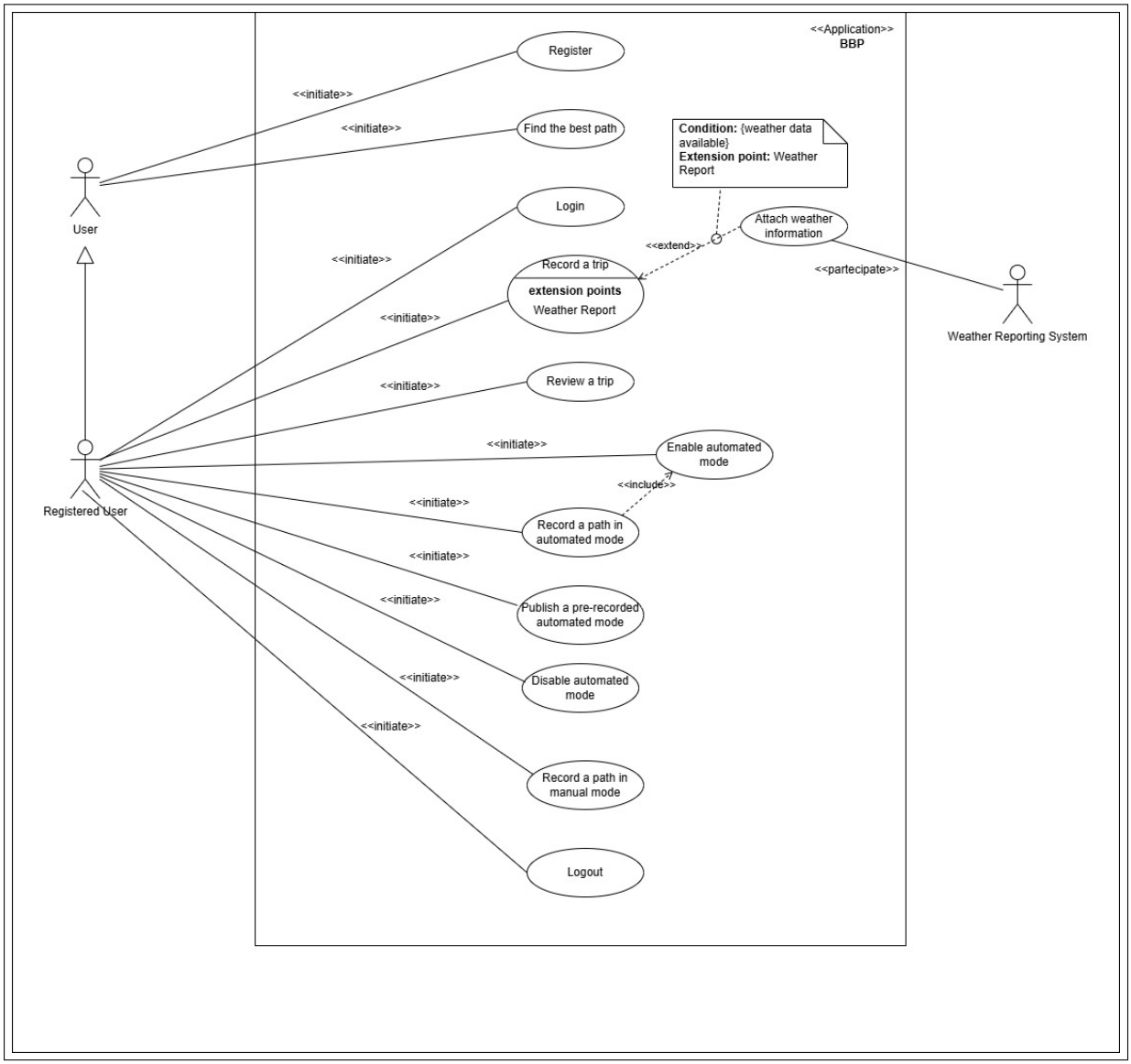


Figure 12: Use Case Diagram

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

3.2.2 Use Cases

[UC1] Register

Name	**Register**
Actors	User
Entry Condition	User opens the BBP platform
Event Flow	1. The user presses "Don't have an account? Sign up" in the first page 2. The system requires the user to fill out a form with the user's username, email address, and a password. 3. The user inserts username, email address, and a password. 4. The user presses the "Sign Up" button 5. The system verifies that: -the username does not exist yet **AND**, -the email address has not been registered yet. 6. The system sends a verification email to the provided address. 7. Outside the system, the user verifies the email address (provided in step 3) by clicking the link inside the email sent in step 6.
Exit Condition	The user has successfully registered to the platform
Exception	E1 - (6) The email cannot be delivered to the provided email address. The system tries to send the email again and displays an error to the user if it fails again E2 - (5) The username already exists. In this case, the system displays an error to the user and asks them to try with another username. E3 - (5) The email already exists. In this case, the system display an error to the user and asks them to try with another email or if they want to login instead.

Table 1: Organization Registration Use Case

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

[UC2] Find the best path

Name	**Find the Best Path**
Actors	User
Entry Condition	One of the following conditions must hold: -User opens the BBP platform **OR** -User is in the home page
Event Flow	1. The user initiates the route search by pressing one of the following buttons: -"Go to the search page" **OR** -"Find your Best Bike Paths" **OR** -"Search" in the navigation bar. 2. The system requires the user to insert an origin and a destination point. 3. The user inserts the origin and destination point. 4. The user presses the "Find routes" button. 5. The system merges information from different users, considering freshness and number of confirming users. 6. The system computes possible bike paths and their score between these points. 7. The system displays paths and their scores in descendant score order.
Exit Condition	The user sees recommended bike paths with visualized scores and information about riding time, total distance and path condition.
Exception	E1 - (6) No valid path between selected points. In this case, the system will provide the user with a blank page and a message saying that there not exists a path between those two points. E2 - (6) System fails to retrieve path data. In this case, the system tries again and, if not possible, warns the user to try again later.

Table 2: Find the best path Use Case

[UC3] Login

Name	**Login**
Actors	Registered User
Entry Condition	Registered user opens the platform
Event Flow	1. The system prompts the registered user to enter email and password credentials. 2. The registered user enters email and password credentials. 3. The registered user presses the "Sign in" button. 4. The system verifies validates entered credentials
Exit Condition	The registered user has successfully logged in to the platform
Exception	E1: (4) - Invalid credentials. In this case, the system displays an error message "Invalid email or password" and prompts the registered user to try again. E2: (4) - Email address not verified. In this case, the system informs the registered user that the email has not been verified and prompts to try again.

Table 3: Login Use Case

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

[UC4] Record a Trip

Name	**Record a trip**
Actors	Registered User
Entry Condition	Registered user is logged in
Event Flow	1. The registered user goes to the registered trips section by: -pressing the "Track your path" button **OR**, -pressing the "Your Trips" button in the navigation bar. 2. The registered user presses the "New Activity" button. 3. The system displays the “New Activity” page, prompting the user to start recording. 4. The registered user presses the "Start recording" button. 5. The system starts recording the trip, collecting GPS location, speed, and sensor data. 6. The registered user can pause or resume the recording. 7. The registered user stops the recording. 8. The system saves the trip information and displays a summary including total time, average speed, and total distance.
Exit Condition	Trip is stored with performance metrics
Exception	E1: (5) - GPS/sensor data not available In this case, the system displays an error to the registered user, describing the problem and suggesting to check the sensors or try again another time. E2: (8) Error saving the data into the system In this case, the system tries to save again until it is successfully saved. E3: (5) The application crashes during the recording In this case, the system pauses the recording and, at the reboot of the application, prompts the registered user to save it or start it again. E4: (4) Permissions not granted The system prompts the registered user for access to the sensors and background execution. If the registered user denies permissions, the system disables automated mode and notifies the registered user.
Extension Point:	Step 8 - Optional: Trigger Weather Report

Table 4: Record a trip Use Case

[UC5] Attach weather information

Name	**Attach weather information**
Actors	Weather Reporting System (WRS)
Entry Condition	A trip without weather data is recorded
Event Flow	1. The system checks the availability of the Weather Reporting System (WRS). 2. The WRS responds with its availability. 3. The system requests contextual weather data for the trip from the WRS. 4. The WRS provides meteorological information, including wind speed, temperature, and weather conditions. 5. The system attaches the received weather information to the trip record.
Exit Condition	Trip is stored with performance metrics and weather information
Exception	E1 - (1) WRS not available. In this case, the system acts like no weather information were available and does not attach anything to the trip. E2 - (3) Weather data not available. In this case, the system does not attach anything to the trip information.

Table 5: Attach weather information Use Case

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

[UC6] Review a trip

Name	**Review a Trip**
Actors	Registered User
Entry Condition	Registered user is logged in
Event Flow	1. The registered user goes to the registered trips section by: -pressing the "Explore your registered trips" button **OR**, -pressing the "Your Trips" button in the navigation bar. 2. The system displays a list of recorded trips, showing the starting and finishing points, and the date of each trip. 3. The registered user presses on a trip. 4. The system displays a detailed view with the information of the selected trip, including all recorded information..
Exit Condition	The registered user reviews trip data.
Exception	E1 - No registered trip associated with the registered user. In this case, the system shows an empty page to the registered user. E2 - Error fetching trip information In this case, the system displays an error message to the registered user, inviting to try again later.

Table 6: Review a trip Use Case

[UC7] Record a Path in Automated Mode

Name	**Record a Path in Automated Mode**
Actors	Registered User
Entry Condition	All of these conditions must hold: -Registered user is logged in **AND**, -Automated mode is enabled.
Event Flow	1. The registered user begins a bike ride with automated mode already enabled -> see Enable Automated Mode 2. The system records GPS, accelerometer, and gyroscope data while the user is cycling. 3. The system detects possible obstacles or potholes from sensor data and saves the information. 4. The registered user stops biking. 5. The system detects when the user ended the bike ride. 6. The system saves the recorded bike path information 7. The system notifies the registered user that a new path has been recorded.
Exit Condition	Path information is stored and is ready for review.
Exception	E1: (2) - GPS/sensor data not available In this case, the system displays an error to the registered user, describing the problem and suggesting to check the sensors or try again another time. E2: (6) Error saving the data into the system In this case, the system tries to save again until it is successfully saved. E3: (2) The application crashes during the recording In this case, the system stops the recording and, at the reboot of the application, acts as if the recording has ended normally.

Table 7: Record a Path in Automated Mode Use Case

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

[UC8] Enable Automated Mode

Name	**Enable Automated Mode**
Actors	Registered User
Entry Condition	Registered user is logged in
Event Flow	1. The registered user presses the toggle button "Automated mode" 2. The system starts collecting data from the device's sensors (GPS, accelerometer, gyroscope) 3. The system notifies the registered user that automatic mode is enabled.
Exit Condition	The system actively collects bike path and sensor data
Exception	E1: (1) Permissions not granted The system prompts the user for access to the sensors and background execution. If the user denies permissions, the system disables automated mode and notifies the user. E2: (2) - GPS/sensor error In this case, the system displays an error to the user, describing the problem and suggesting to check the sensors or try again another time.

Table 8: Enable Automated Mode Use Case

[UC9] Disable Automated Mode

Name	**Disable Automated Mode**
Actors	Registered User
Entry Condition	All of these conditions must hold: -Registered user is logged in **AND**, -Automated mode is enabled.
Event Flow	1. The registered user presses the toggle button "Automated mode" 2. The system stops collecting data from the device's sensors (GPS, accelerometer, gyroscope) 3. The system notifies the registered user that automatic mode is disabled.
Exit Condition	System is no longer collecting bike path and sensor data
Exception	E1: (1) Trip/Path recording in progress In this case, the system stops recording the trip/path. E2: (2) - GPS/sensor error In this case, the system displays an error to the registered user, describing the problem and suggesting to check the sensors or try again another time.

Table 9: Disable Automated Mode Use Case

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

[UC10] Publish a pre-recorded automated mode path

Name	**Publish a pre-recorded automated mode path**
Actors	Registered User
Entry Condition	Registered user is logged in, automated mode is enabled, and a path has been recorded
Event Flow	1. The system notifies the registered user that a new path has been recorded and is ready to be reviewed. 2. The user accesses the recorded path by either pressing the notification **OR** opening the application after ending the recording. 3. The system displays the recorded path information 4. The system prompts the registered user to either confirm, edit, or delete the recorded path information. 5. The registered user reviews and, if necessary, edits the path information. 6. The registered user presses "Save and publish". 7. The system saves the path information in the central database, making it available for other users.
Exit Condition	Path information is stored and is published
Exception	E1: (4) The registered user deletes the recording. In this case, the recording is discarded. E2: (7) Error saving the data into the system In this case, the system tries to save again until it is successfully saved.

Table 10: Publish a pre-recorded automated mode path Use Case

[UC11] Record a Path in Manual Mode

Name	**Record a Path in Manual Mode**
Actors	Registered User
Entry Condition	Registered user is logged in
Event Flow	1. The registered user goes to the registered trips section by: -pressing the "Track your path" button **OR**, -pressing the "Track" button in the navigation bar. 2. The system prompts the registered user to provide path information via a form. 3. The registered user fills out the form with the streets and intersections forming the path, specifying the status of each road and detailing any issues if the road is not optimal. 4. The registered user presses "Save and publish" 5. The system saves the path information in the central database, making it publicly available for other users.
Exit Condition	Path information is stored and can be made publishable
Exception	E1: (3) Mandatory data not provided In this case, the system displays a message to the registered user, prompting them to add the missing details. E2: (7) Error saving the data into the system In this case, the system tries to save again until it is not saved. E3: (4) The registered user deletes the entry In this case, the system discards the path.

Table 11: Record a path in manual mode Use Case

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

[UC12] Logout

Name	**Logout**
Actors	Registered User
Entry Condition	Registered user is logged in.
Event Flow	1. Registered user presses "Logout" on the navigation bar. 2. The system prompts the registered user to confirm the logout action. 3. The registered user confirms. 4. The system revokes the user's session credentials and clears all session-related data. 5. The system redirects the user to the login page.
Exit Condition	Registered user is no longer authenticated; session is closed
Exception	E1: (4) Error to terminate session correctly. In this case, the system retries to terminate session correctly until it is possible. E2: (2) The registered user cancels the logout action. In this case, the system redirects the registered user to the home page.

Table 12: Logout Use Case

3.2.3 Mapping

Goal	**Requirements and Domain Assumptions**	**Use cases**
G1: Users can register and have a profile in BBP application.	**R1**: The system must allow users to register using a username, an email and a password. **R2**: The system must verify that credentials are unique. **R3**: The system must send a verification email to the user. **R4**: The system must let people verify their email after receiving the verification request. **R5**: The system must activate accounts only if verified. **R6**: The system must allow registered users to log in using email and password. **R7**: The system must verify the correctness of the credentials. **R8**: The system must allow registered users to log in only if the credentials are correct. **R9**: The system must allow registered users, that logged-in, to log out. **R10**: The system must delete user's session. **D2**: The internet connection remains relatively stable throughout the session. **D13**: Users will give non-misleading information about their identity.	**UC1**: Register **UC3**: Login **UC12**: Logout

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Goal	Requirements and Domain Assumptions	Use cases
G2: Users manually create bike paths in an inventory by specifying the name of the streets and the status.	**R23**: The system must allow registered users to manually insert path information using a form. **R24**: The system must require the user to specify roads and intersections in the path. **R25**: The system must allow registered users to specify the condition of each road in the path. **R26**: The system must allow registered users to report additional issues (f.e. potholes). **R27**: The system must save the path information in the central database. **D2**: The internet connection remains relatively stable throughout the session. **D4**: Users are expected to provide accurate and non-misleading information when creating new paths. **D5**: Users do not use the application or the platform for purposes unrelated to biking activities or the exploration of new routes as a recreational activity. **D8**: The date related to the new inserted path is correct. **D9**: The app is used where bike track already exists. **D10**: The biking paths have little to no cars presence. **D11**: The biking paths have speed limits compatible with the average speed of a bike.	**UC11**: Record a path in manual mode

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Goal	Requirements and Domain Assumptions	Use cases
G3:Users create bike paths in an automated mode where the system acquire data from the user's mobile device and reconstruct the path.	**R13**: The system must ask registered users to give permissions for the sensors. **R14**: The system must read the sensors data only if the permissions are given by the registered user. **R28**: The system must let registered users enable automated mode. **R29**: The system must let registered users disable automated mode. **R30**: The system must let registered users add path information via automated mode. **R31**: The system must record data using sensors only if automated mode is enabled. **R32**:The system must recognize if the registered user is biking. **R33**: The system must start recording the path only if the system recognizes that the registered user is biking. **R34**: The system must use the sensors' data to record the path. **R35**: The system must record intersections and road crossed by the registered user. **R36**:The system must recognize obstacles in the path (f.e. pothole) using sensors' data. **R37**: The system must recognize if the registered user stopped biking. **R38**:The system must stop recording the path if the system recognizes that the registered user stopped biking. **R39**: The system must save the recorded bike path information in "to be reviewed" state after an automated mode recording **R40**: The system must notify the user that a path has been recorded this way. **R41**: The system must require registered users to review and edit information collected via automated mode before publishing. **R42**: The system must allow registered users to discard the information collected via automated mode. **R43**: he system must allow registered users to save and publish the information recorded via automated mode.	**UC6**: Enable automated mode **UC7**: Record a path in automated mode. **UC8**: Publish a pre-recorded automated mode path. **UC9**: Disable automated mode.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

	R27: The system must save the path information in the central database.	
	D1: Each user rides a bicycle while the application runs in background in case of automated mode.	
	D2: The internet connection remains relatively stable throughout the session.	
	D3: Users are willing to review and refine the automatically collected information gathered by the device in the automated-mode.	
	D4: Users are expected to provide accurate and non-misleading information when creating new paths.	
	D5: Users do not use the application or the platform for purposes unrelated to biking activities or the exploration of new routes as a recreational activity.	
	D6: Third-parties services provide reliable information to the application.	
	D7: Hardware devices must provide reliable sensor data to the system to let it acquire the right performance data.	
	D8: The date related to the new inserted path is correct.	
	D9: The app is used where bike track already exists.	
	D10: The biking paths have little to no cars presence.	
	D11: The biking paths have speed limits compatible with the average speed of a bike.	
	D12: Users will stop cycling when they want to end the recording of the path.	

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Goal	Requirements and Domain Assumptions	Use cases
G4: Users browse bike paths within the inventory. They can specify an origin and destination and ask the system to visualize the bike path on a map.	**R44**: The system must allow every user, regardless of their registration status, to browse for paths. **R45**: The system must allow the user to fill up a form regarding an origin and destination point. **R46**: The system must compute the best path(s) considering the specified points. **R50**: The system must show the user all the possible best paths in descendant score order, with their scores and metrics. **R51**: The system must show the user a map of the best path(s). **D2**: The internet connection remains relatively stable throughout the session.	**UC2**: Find the best path.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Goal	Requirements and Domain Assumptions	Use cases
G5: Users record their trips and store them to keep track of their cycling activities.	**R13**: The system must ask registered users to give permissions for the sensors. **R14**: The system must read the sensors data only if the permissions are given by the registered user. **R15**: The system must allow registered users to record their bike trips. **R16**: The system must use sensors to track the biker's trip and their performance metrics. **R17**: The system must allow the registered user to save the trip recorded. **D2**: The internet connection remains relatively stable throughout the session. **D5**: Users do not use the application or the platform for purposes unrelated to biking activities or the exploration of new routes as a recreational activity. **D7**: Hardware devices must provide reliable sensor data to the system to let it acquire the right performance data. **D8**: The date related to the new inserted path is correct. **D9**: The app is used where bike track already exists. **D10**: The biking paths have little to no cars presence. **D11**: The biking paths have speed limits compatible with the average speed of a bike.	**UC4**: Record a trip.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Goal	Requirements and Domain Assumptions	Use cases
G6: Users can see for a recorded trip various statistical data of the user performances as well as meteorological information during the respective trip.	**R18**: The system must check the availability of the Weather Reporting Service (WRS) before requesting data. **R19**: The system must ask the WRS for weather information regarding the trip considered. **R20**: The system must attach weather information to the trip entry, if available. **R21**: The system must show registered users a list of their recorded trips. **R22**: The system must allow registered users to see details of a specified recorded trip. **D6**: Third-parties services provide reliable information to the application. **D8**: The date related to the new inserted path is correct.	**UC5**: Attach weather information. **UC6**: Review a trip.

Goal	Requirements and Domain Assumptions	Use cases
G7: The system uses information published by the users about the paths by merging pieces of information to generate the optimal path, using the freshness and number of confirmed data.	**R47**: The system must merge information from multiple users regarding a road, considering freshness of information and number of confirming users. **R48**: The system must compute an average status for the specified path. **R49**: The system must compute a score for the specified path, considering their status and the easy reachability. **D2**: The internet connection remains relatively stable throughout the session. **D4**: Users are expected to provide accurate and non-misleading information when creating new paths. **D8**: The date related to the new inserted path is correct.	**UC2**: Find the best path.

3.2.4 Sequence Diagrams [4]

This subsection presents notifications modeled at a high level to illustrate user interactions. It deliberately omits detailed design, architectural considerations, and synchronization aspects to focus on the conceptual interaction flow rather than implementation specifics.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

[UC1] Register

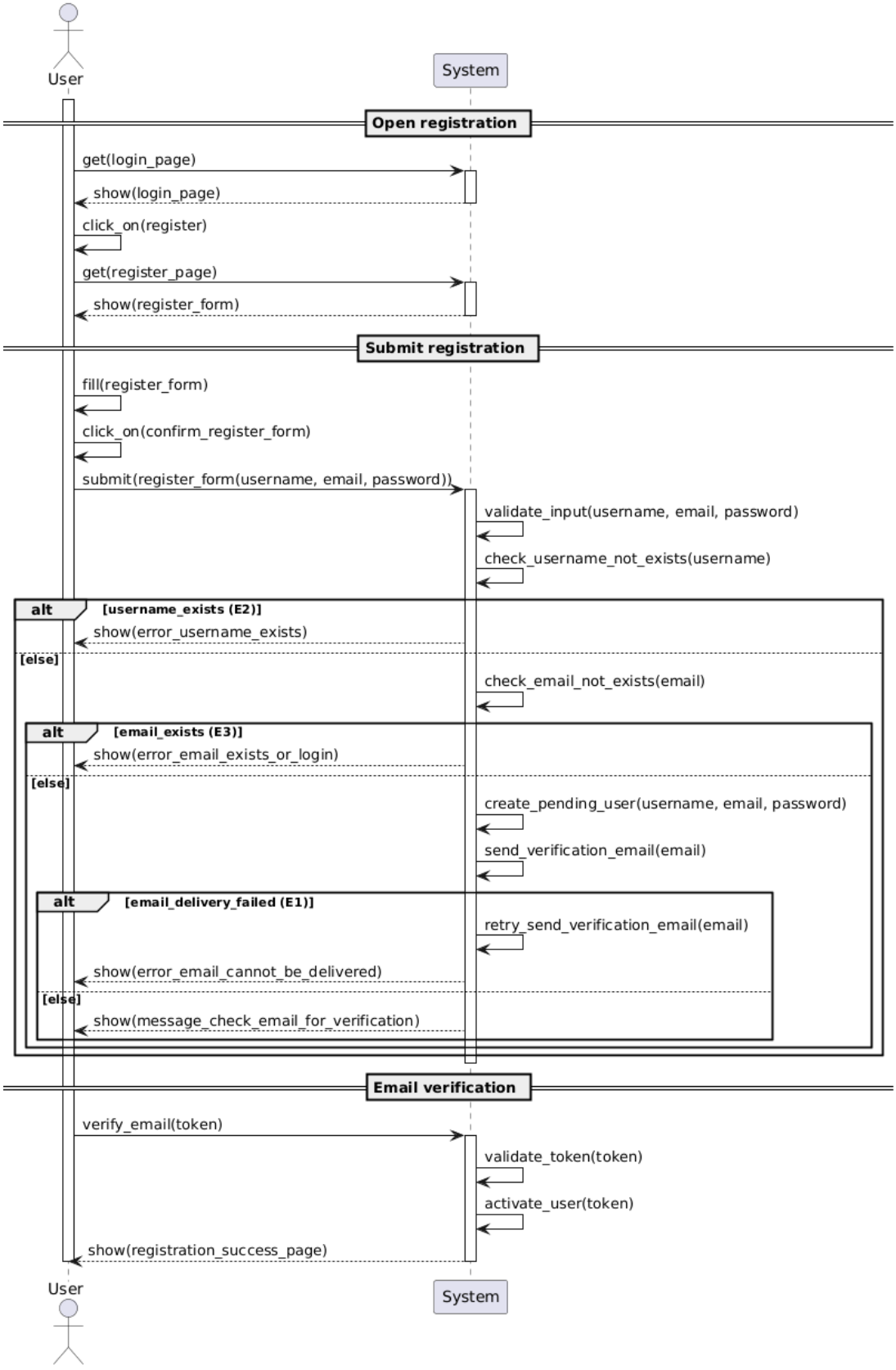


Figure 13: Sequence diagram for register.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

[UC2] Find the best path

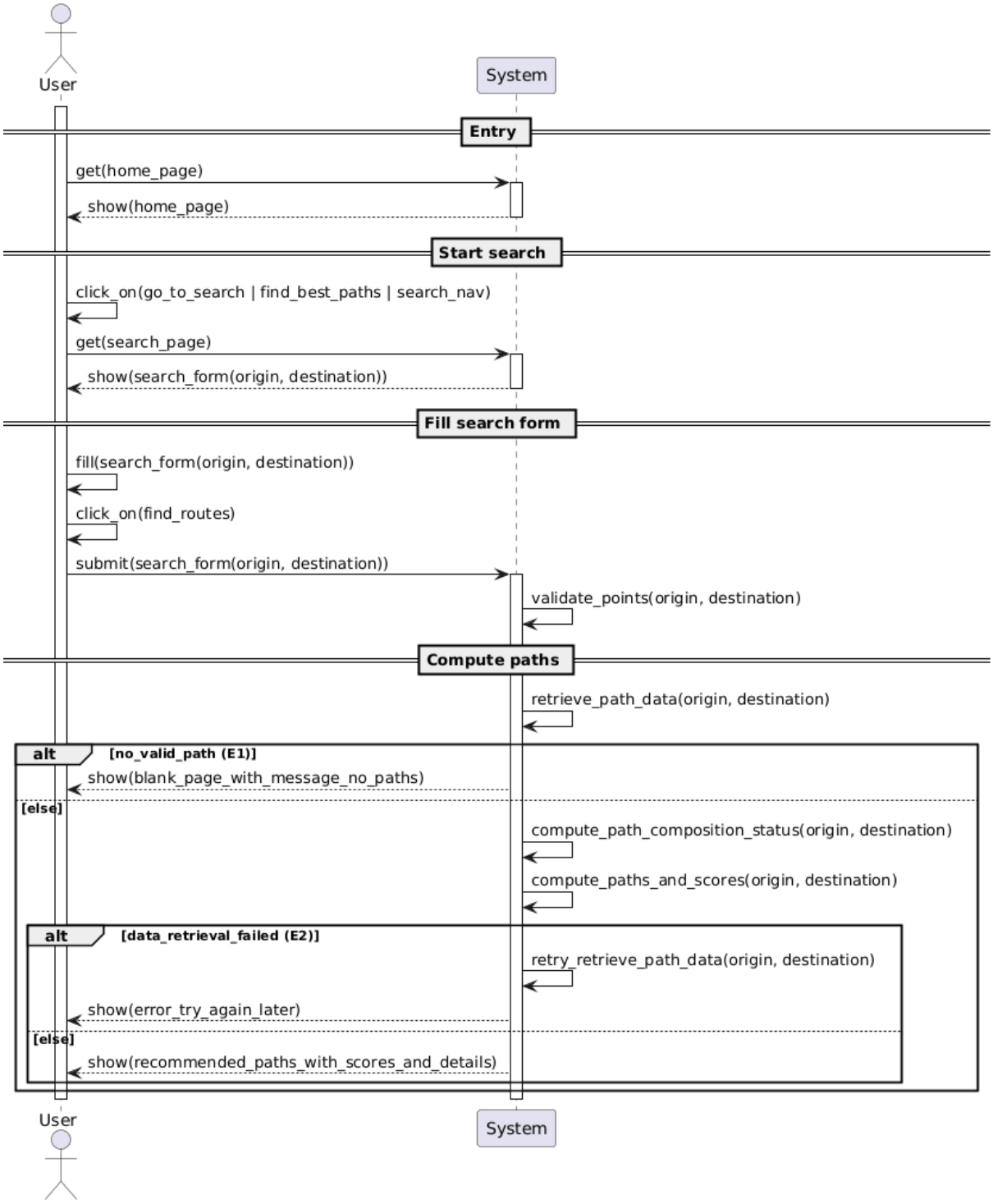


Figure 14: Sequence diagram for find best path.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

[UC3] Login

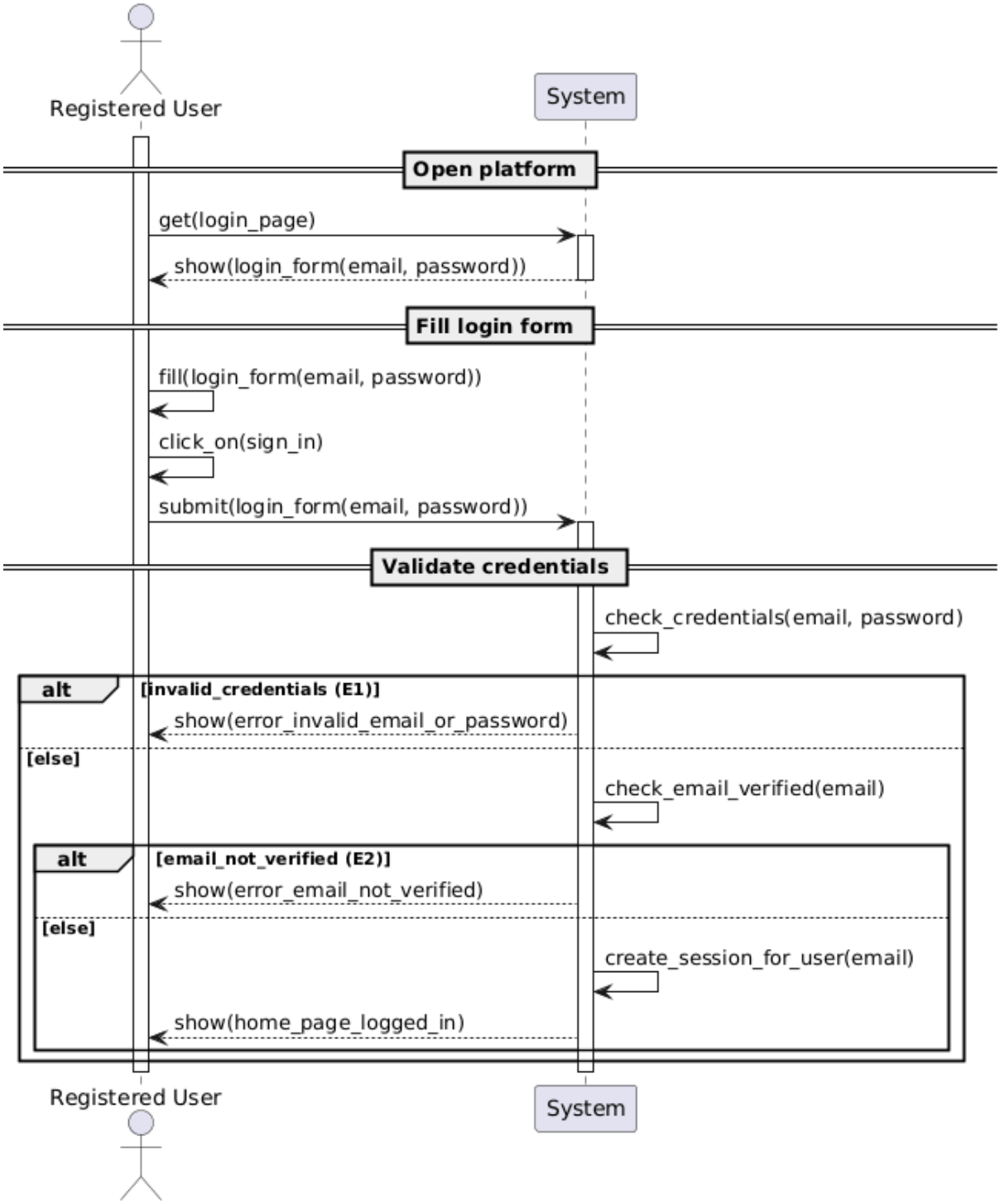


Figure 15: Sequence diagram for login.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

[UC4] Record a Trip

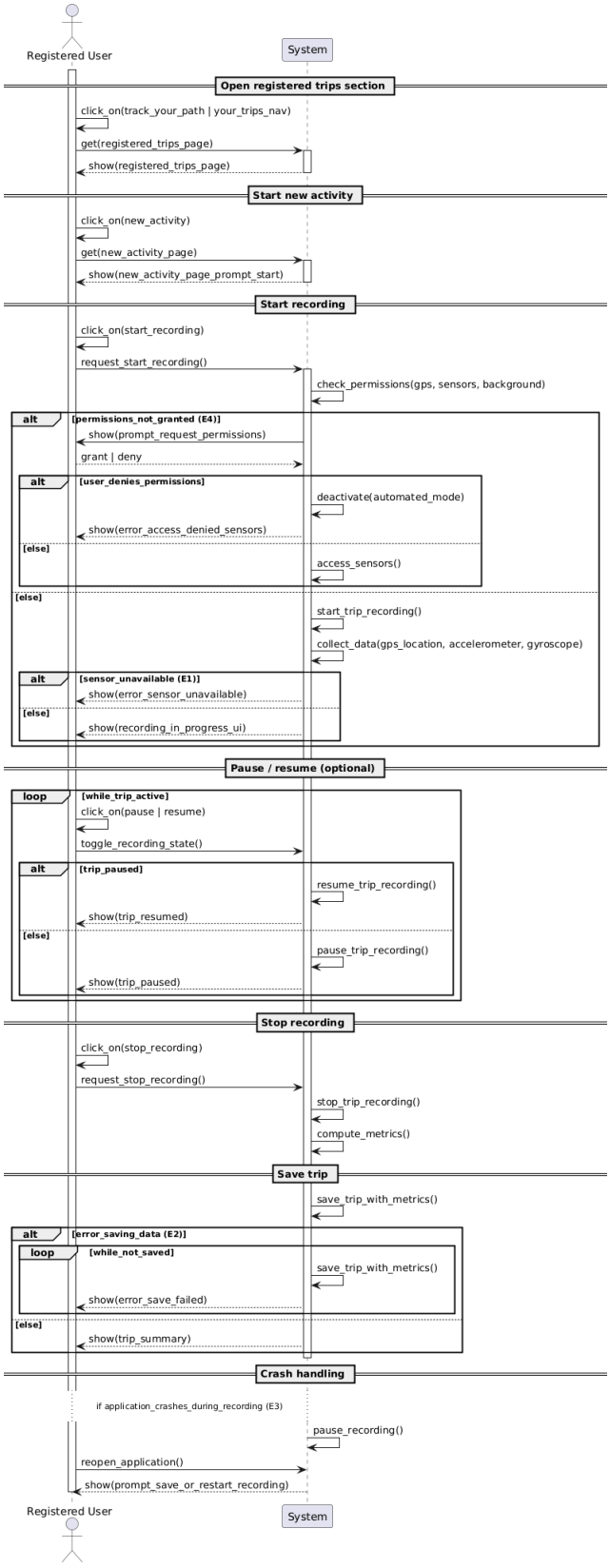


Figure 16: Sequence diagram for record trip.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

[UC5] Attach weather information

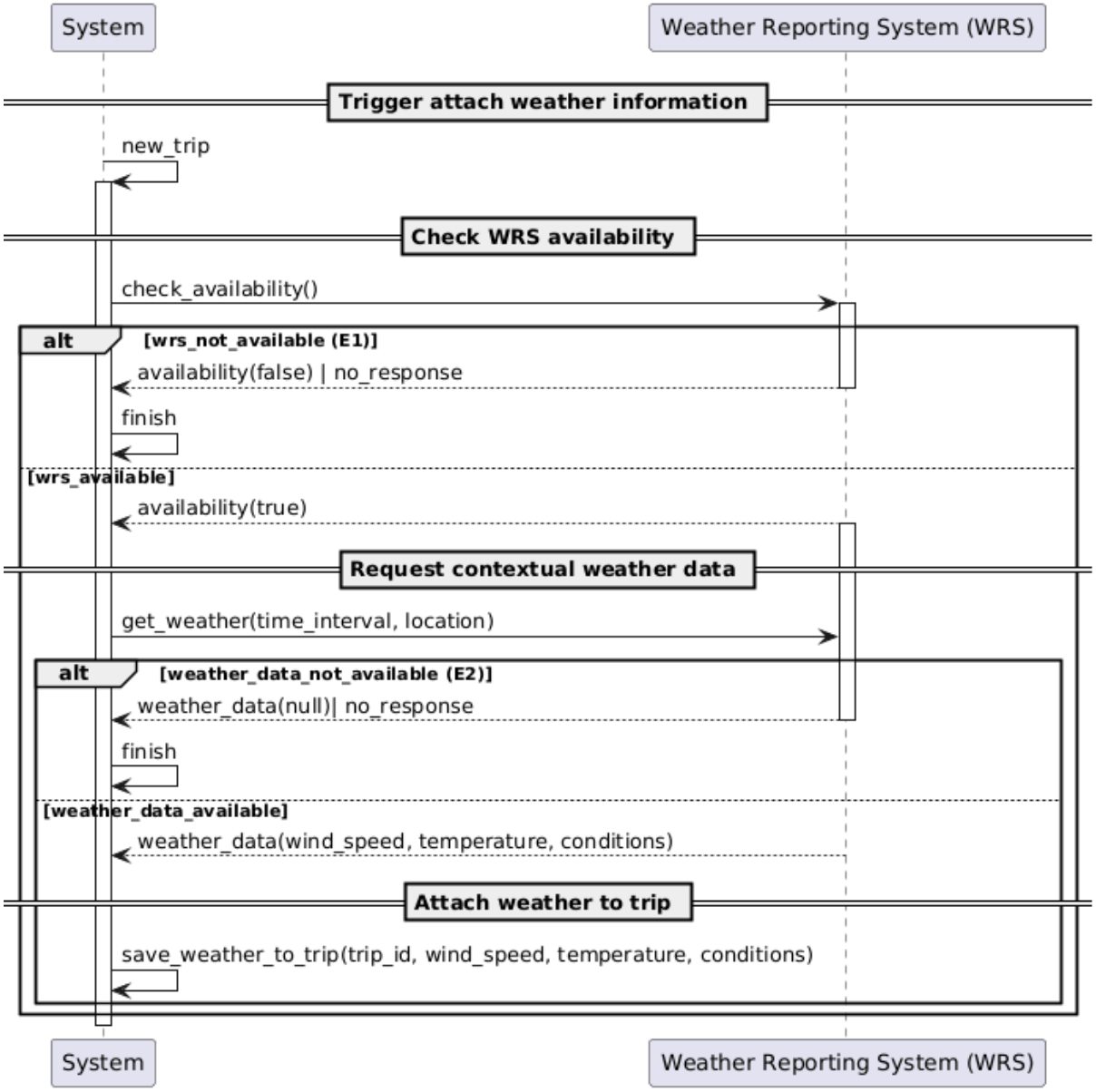


Figure 17: Sequence diagram for attach weather information.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

[UC6] Review a trip

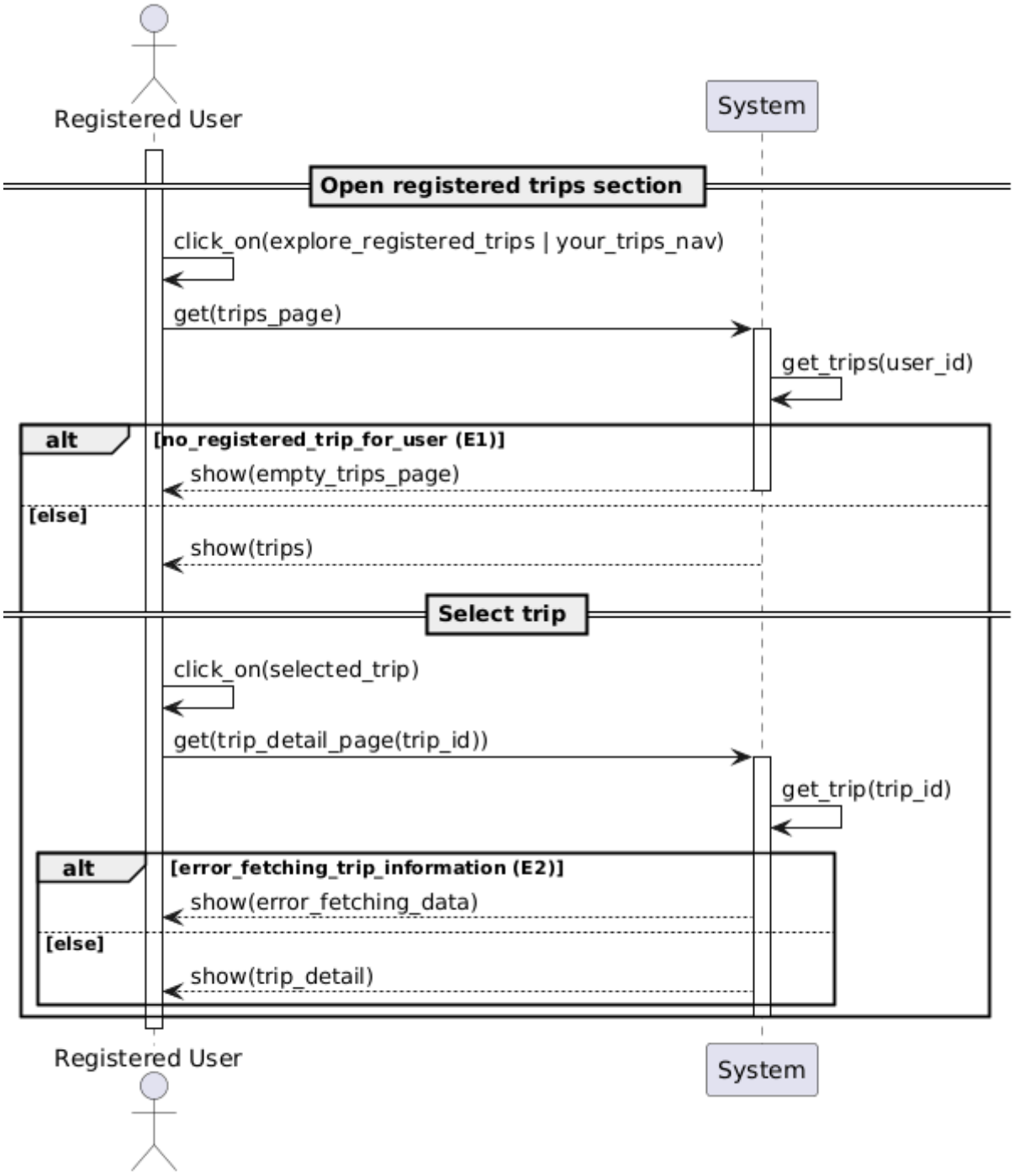


Figure 18: Sequence diagram for review trip.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

[UC7] Enable Automated Mode

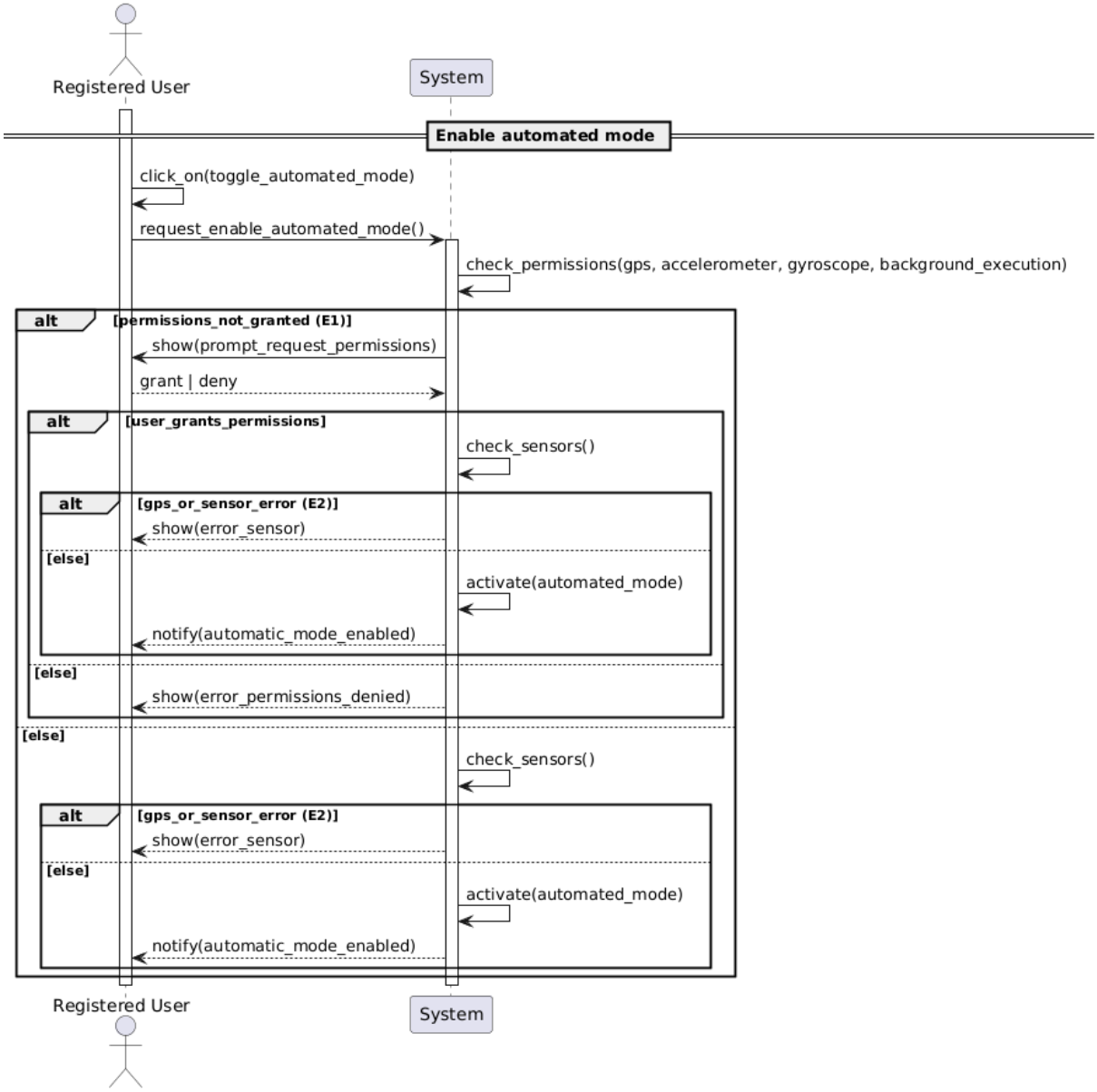


Figure 19: Sequence diagram for enable automated mode.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

[UC8] Disable Automated Mode

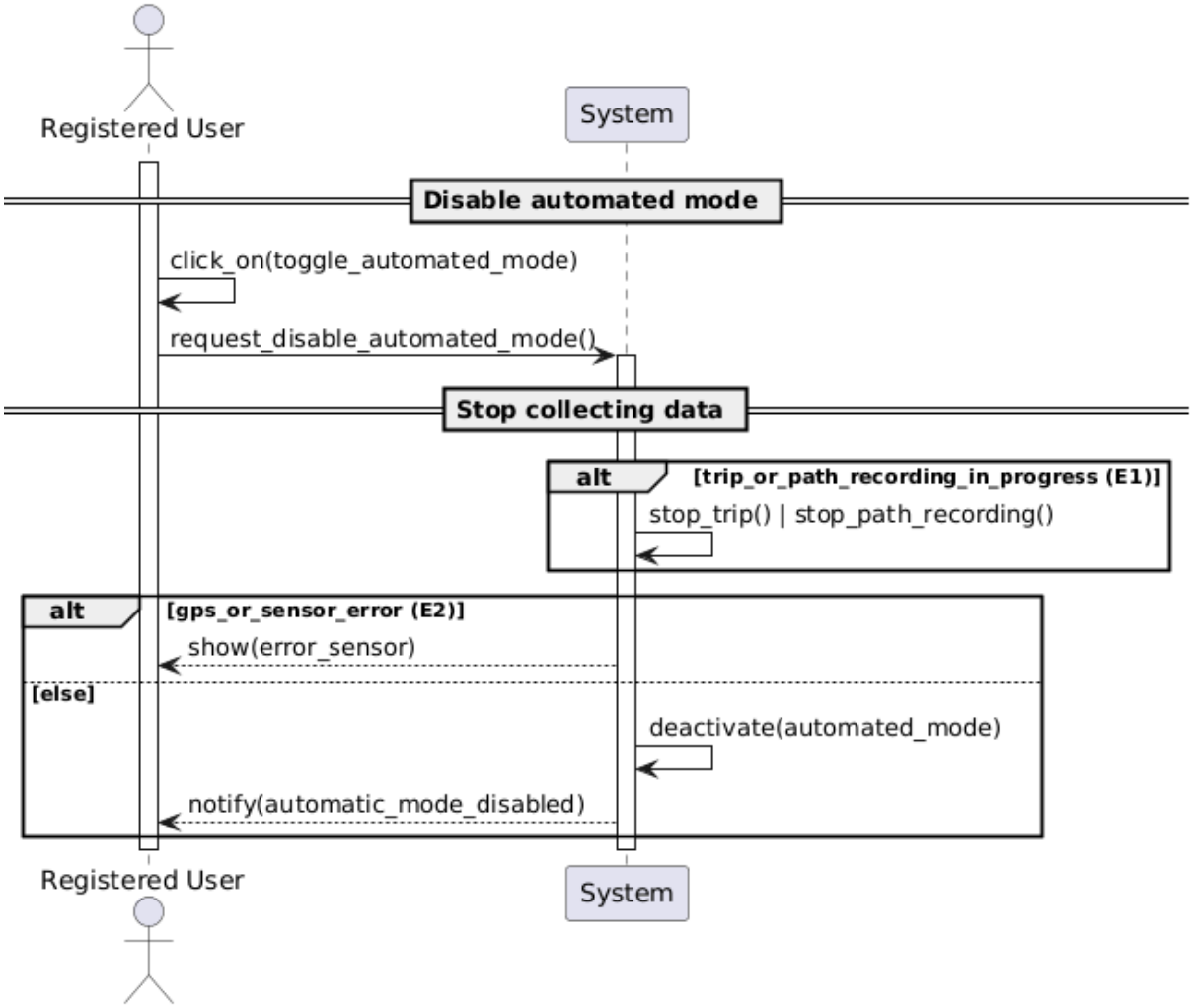


Figure 20: Sequence diagram for disable automated mode.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

[UC9] Record a Path in Manual Mode

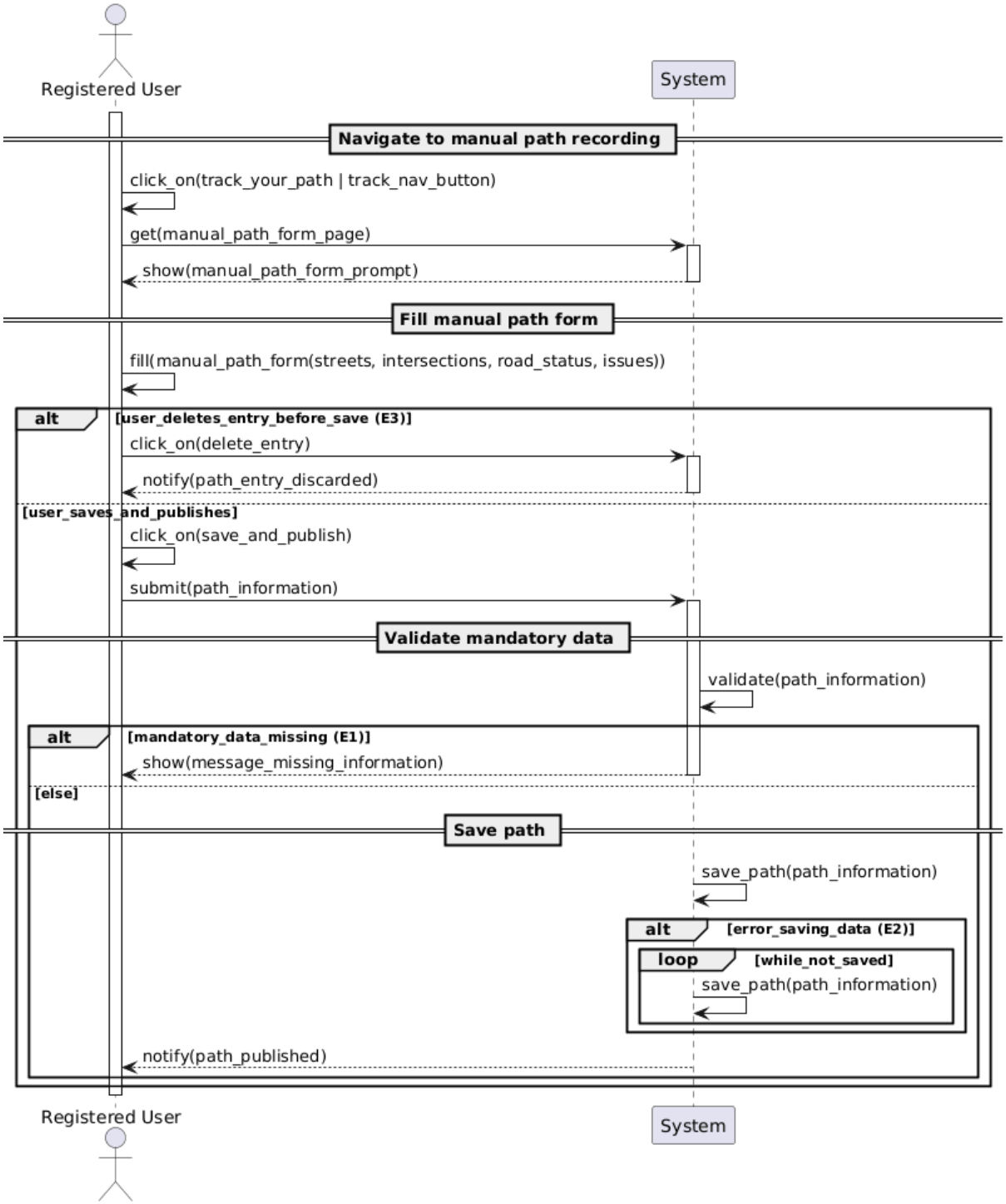


Figure 21: Sequence diagram for record a path in manual mode.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

[UC10] Record a Path in Automated Mode

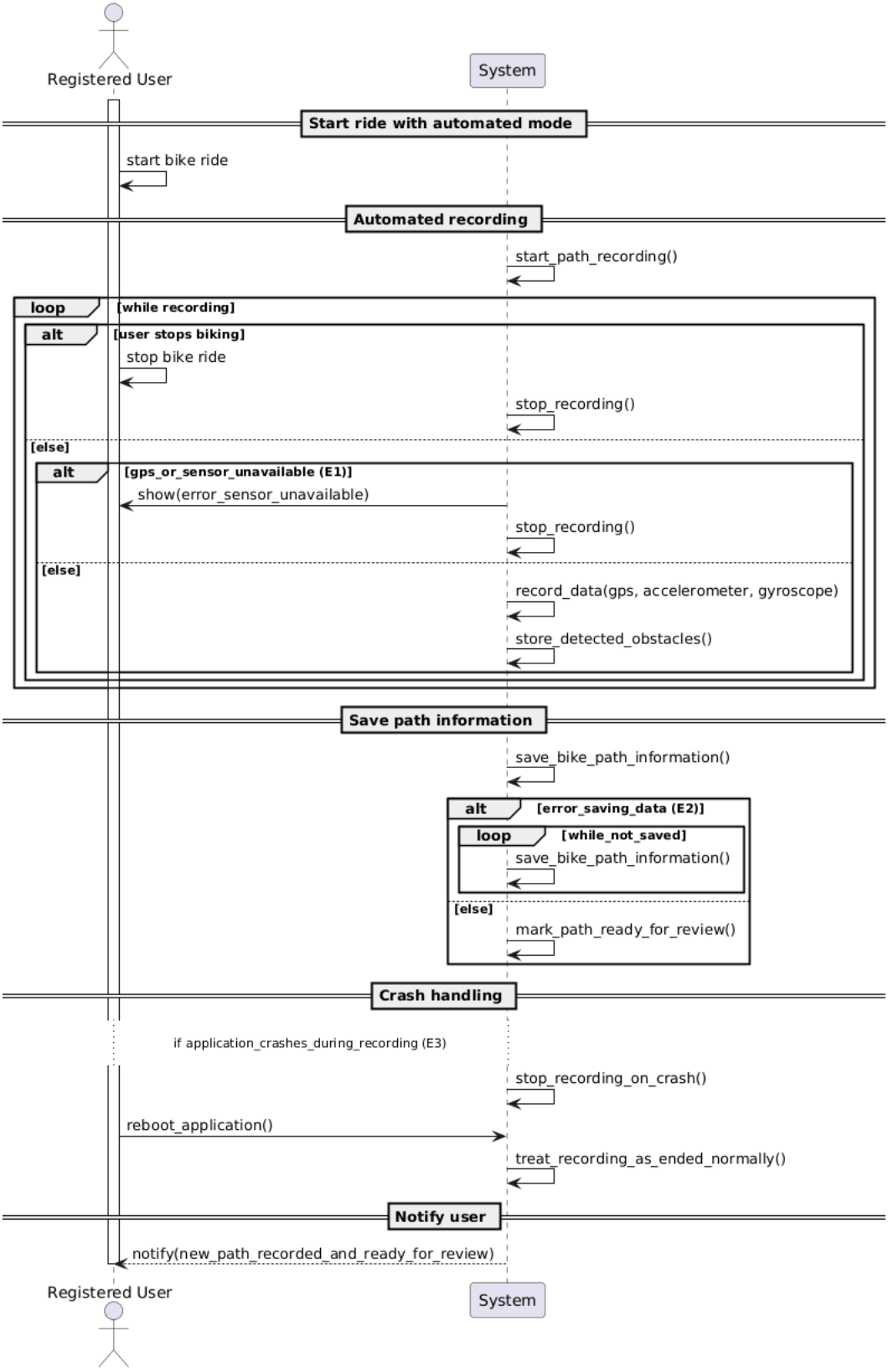


Figure 22: Sequence diagram for record a path in automated mode.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

[UC11] Publish a pre-recorded automated mode path

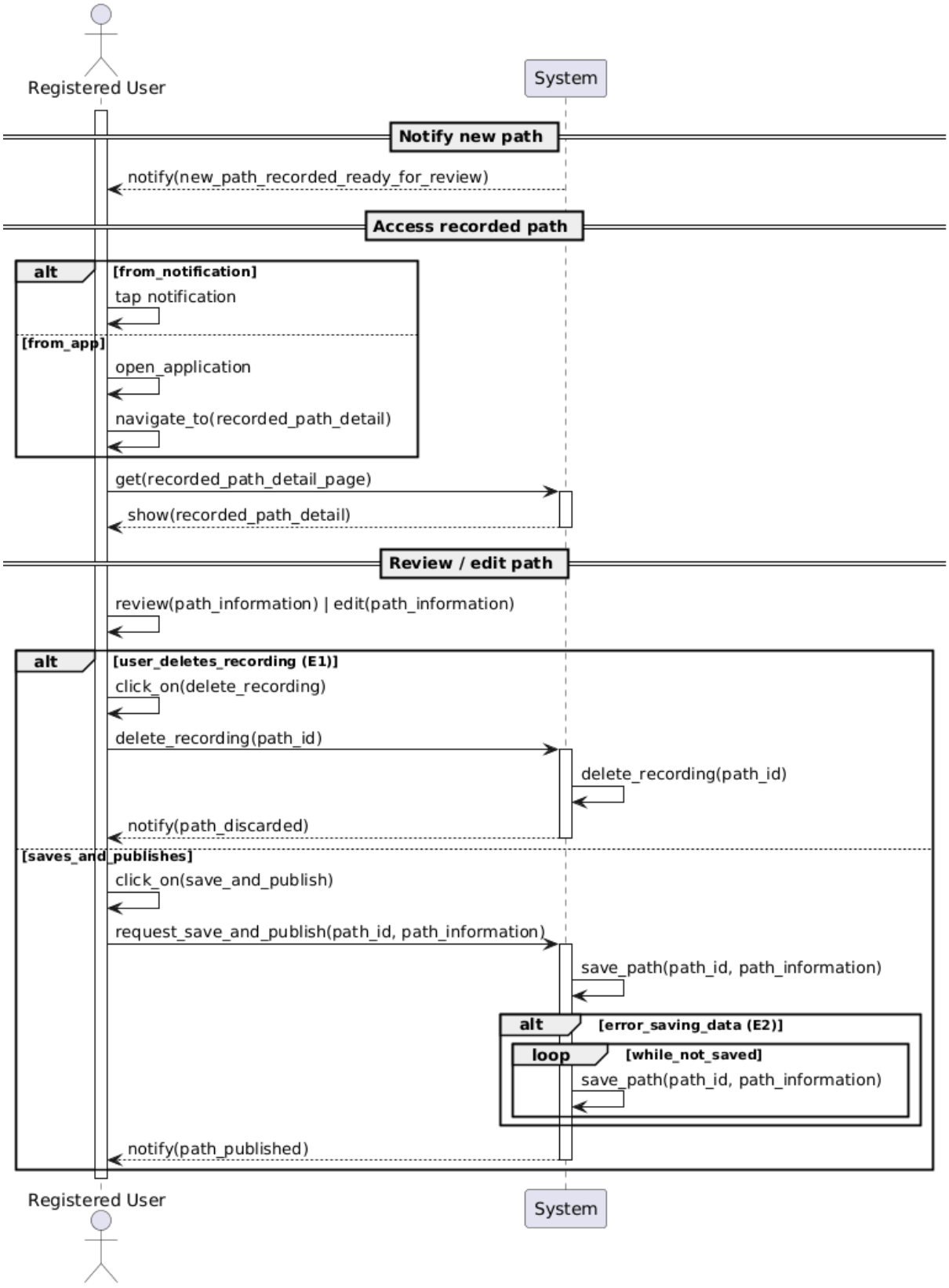


Figure 23: Sequence diagram for publish a pre-recorded automated mode path.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

[UC12] Logout

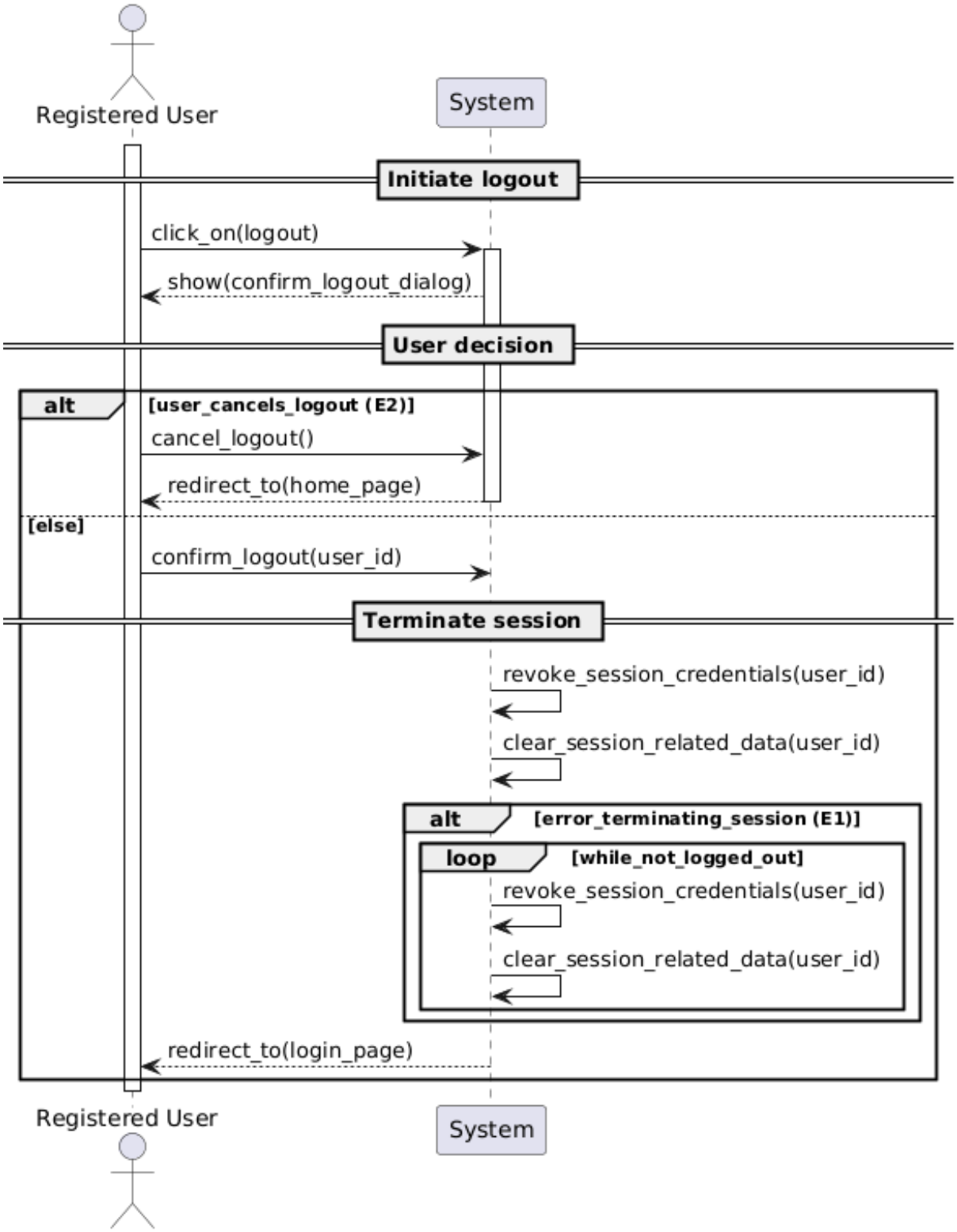


Figure 24: Sequence diagram for logout.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

3.3 Performance Requirements

The system must be designed to handle a large number of concurrently connected and registered users, making scalability one of its primary functional requirements. As the platform is expected to serve many users at a time from many devices, it must be able to expand seamlessly while keeping performance degradation to a minimum. In other words, despite the number of users that use the different functionalities of the System, this still has to be reliable and stable.

Availability is equally essential: due to one of the functionality of the system being a "searching for paths" one, the uptime must be maximized and deliver information rapidly and efficiently. Key features such as search, tailored paths, and publishing processes must guarantee low response times and avoid introducing operational bottlenecks in such events, especially at peak of requests and high number of paths submitted. In addition, the system must support the upload and management of substantial volumes of metadata of the user paths, without compromising responsiveness or stability in the publish processing, but even when the automated mode is activated and user (taken account it is registered) is currently riding.

Furthermore, any possible estimation of active users on the platform is hard to get, as there are many competitors that deals only with specific functionalities of BBP. To give an example, if we consider Strava's platform that deals with trip metrics, there are estimation of around 40-50 millions monthly users [12]. So, as a simple comparison (that takes account the part that gather trip information), BBP is supposed to cope with a number of concurrent users around 30.000, assuming an hypothetical average of two hours of usage per access. However, it is also reasonable to state that this estimation does not take account of the variance of such values, as there might be periods of the year with much greater accesses, and others with smaller ones. As a result, usage cannot be considered exclusively stable but, instead, a value that might oscillate.

3.4 Design Constraints

3.4.1 Standards compliance

Conformity Standards

1. GDPR: The system should manage user data, location tracking and data path in the European Union, thus it is compliant with the General Data Protection Regulation according to the European guidelines.
2. Local privacy laws: The System should manage user data, location tracking and data path, thus it is compliant with the local laws in countries outside the European Union.
3. ISO/IEC 27001 [9]: The system must adopt structured and comprehensive information security management, in order to protect sensitive information and be adaptable to new risks.
4. ISO/IEC 25010 [10]: The system must be developed as to follow proper quality standards for reliability, performance, maintainability, and security.

Development Standards

1. ISO/IEC 12207 [8]: The system must apply the best practices in all the main processes that bring to its realization, in full compliant with the software life-cycle management standards.

3.4.2 Hardware Limitation

1. In order for the application to work properly, the devices must be able to allocate enough resources to the background processes. In other words, a good CPU and enough RAM are needed. Minimum requirements

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

like a quad-core processor is needed as a reliable resource for background activities. Furthermore, 4 GB of Ram is needed but for multi-tasking processes more of that is suggested. Long paths, especially for those taking hours, might affect the battery life-cycle of the device, especially if the automated mode is activated, as the usage of sensors is highly expensive in terms of battery consumption. As a result, battery devices shall be recent or have high volume capacity in order to avoid any early shutdown, leading the device to shut down the ongoing path data collection.

2. Sensors, especially for the automated mode, are strictly necessary such as, gyroscopes, GPS sensors or even GNSS equivalent and accelerometer. Drivers shall be updated in order to use properly those sensors.
3. In terms of software requirements, devices must provide the rights to the application to use most of the features in background in order to collect data of the path, as mentioned before. As a result, only latest versions of OS should be considered. However, as minimal requirements, in terms of Android Environment, require Android 10, that supports foreground-services and location authorizations in background, as might be limited in previous versions. For similar reasons, even in Apple environment, a minimum iOS version must be considered, by considering iOS 13, as it provides improved location tracking in background.
4. In terms of data collection, the System collects all the user data and the related path. As a result, the System should be able to provide consistency and reliability in terms of paths scores and analysis. This is because, a good DBMS, especially an hybrid one, provide a good mixture of qualities of relational and non-relational models, as the System has to deal with different types of data, and concurrently also provide flexibility and high responsiveness.

3.4.3 Any other constraint

In terms of connectivity, devices should rely on wireless infrastructures in order to send and receive data in case of collection of data, search and publication of paths. So, this is mostly an external party that affects, however, the user experience of the application as an overall.

3.5 Software System Attribute

The BBP system must satisfy several key software attributes to ensure reliability, usability, and overall performance.

3.5.1 Scalability

The most critical element is scalability as the system must efficiently handle a large number of registered and anonymous users simultaneously. In fact, the most costly effort is to decrease as much as possible the performance degradation even under peak usage. The system should manage new user registration and any surge of signs up. So the systems as a overall is supposed to be well efficiently structured.

3.5.2 Availability

As a result of Scalability, Availability is even crucial. The Search mode for bike paths requires to handle many sets of data already stored and organized in schema by the DBMS. As a result, the system must remain responsive and accessible at all times, delivering information quickly even at the highest peak of usage. Not to mention that the system must also provide robustness and fault tolerance. So, the system has to ready and able to manage many accesses at a time. If we consider periods of time when weather is especially suitable for riding, we expect to have peak of usage in the late morning (9:00 - 12:00) and late afternoon (17:00 - 20:00) between April and September. BBP is expected to be available, accessible and ready.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

3.5.3 Reliability

The system should rely on a robust backup and recovery infrastructure capable of preserving data integrity at all times, even in the presence of unexpected faults. All critical elements must exhibit high reliability, so the architecture should incorporate redundant and parallelized components, ensuring that the failure of a single node, service, or storage unit does not compromise the availability of the platform.

3.5.4 Security and privacy

Security is particularly important in terms of paths that are not directly related to the users, once published. Sensitive information must be stored and transmitted securely respecting user anonymity. In fact, data must be well stored and restricted accessed, preventing backdoor access and similar events. Furthermore, as GPS location is essential in the automated mode, user privacy is one of the most important aspect to mention. Given the sensitive nature of the information handled such that, every piece of user-related data must be protected with modern, robust cryptographic techniques. Only algorithms that are still considered secure by current standards should be used, such as hashing functions with adequate resistance like SHA-256, and encryption schemes such as AES-256 to safeguard stored content.

3.5.5 Maintainability

In order to update the system in future and take account of additional new features, maintainability is then crucial, as well as an structured modular architecture. Furthermore, even a documentation for the related code should be clear, complete and easy to those in charge to maintain the system in future. Additionally, the project should adopt standard development practices such as version control workflows (such as Git flow), automated testing, continuous integration/continuous delivery (CI/CD), and coding style guidelines. These practices reduce technical debt, improve code quality, and make sure that updates can be introduced safely and efficiently.

3.5.6 Portability

The system should be designed to be accessible and installed on operating systems such as iOS and Android. As a result, all the designed features should be all available, independently on the OS been used. Finally, the installation and update process should follow each platform's best practices, ensuring seamless deployment via official app stores, secure updates, and minimal friction for the user.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

4 Formal Analysis Using Alloy

4.1 Alloy code

G1 — Users can register and have a profile in BBP

This goal ensures that the system correctly manages user identities and registration. The assertion *differentUsersCannotBeSameUser* checks the uniqueness of users, stating that two distinct user instances cannot represent the same logical user. This prevents identity collisions within the system.

The assertion *userRegisteredNotAnonymous* verifies that every registered user is not anonymous. In other words, once a user is registered, both must be associated with a concrete and identifiable profile. Moreover, these assertions guarantee that users in BBP are uniquely identifiable, non-anonymous once registered, and consistently represented throughout model phases.

G2 — Manual path creation

This goal concerns the manual creation of paths by users. The predicate *showPath* models the evolution of the system state. So, initially, no trips are present. Eventually, two trips exist, representing the successful creation and storage of paths. This temporal behavior validates that manual path creation leads to persistent system state changes, confirming the goal. Trips are chosen on the select phase.

G3 — Automatic path creation

This goal ensures that automatic and manual path creation are mutually exclusive. The assertion *AutomatedXorManual* expresses an 'exclusive-or' between the two modes. Specifically, if a data w is equal to Automated and at the same time equal to *p.Recorded*, then it must not be equal to Manual. Furthermore, if w is equal to Manual and equal to *p.Recorded*, then it must not be equal to Automated. This allows that a recorded path cannot either automatically and manually created at the same moment. The Alloy analyzer confirms this behavior by finding no counterexamples, meaning the constraint is consistent.

G4 — Browsing and viewing paths

The function *searchPath* retrieves all paths composing a given trip, that it is constrained strictly to 1 by the facts. The predicate *selectPath* calls the function and it ensures that at least one path is available for selection. This confirms that the system can search, retrieve, and select paths corresponding to user-defined criteria, fulfilling the browsing requirement.

G5 — Recording and storing trips

The function *extractTrip* ensures that each user is associated with at most one trip at a time. The predicate *viewTrip* models the transition from no trips to one trip. These constraints validate that trips are correctly going through the phases, uniquely associated with users, and persist in the system.

G6 — Trip statistics and weather conditions

The predicate *viewTripAtCheckInfo* ensures that a trip eventually exists. Then the trip reaches the *CheckInfo* situation. The trip belongs to the requesting user. This verifies that statistical and contextual information is available and accessible for recorded trips.

G7 — Merging information and generating the optimal path

This goal describes the final phases of application usage, including information merging and path publication. The predicate *show[]* models the evolution of the system over time. Initially, the number of Trip instances is 0, representing the state before any trip is created. At some intermediate point, the number of Trip instances becomes

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

3, representing multiple partial or candidate trips during processing. Finally, there exists a Trip whose situation is *Publish*, representing the successful generation and publication of the optimal path. Overall, this goal captures the intended steps of the application, from initial usage, through intermediate processing steps, to the final publication of a single optimal trip of a given user.

```
1
2  abstract sig Status {}
3  one sig Optimal, Acceptable, Sufficient, Difficult extends Status {}
4  abstract sig Way{}
5  one sig Automated, Manual extends Way{}
6  sig idUser {}
7  sig idPath{}
8  sig idTrip{}
9
10 enum currentSituation {
11                 Search,
12                 Select,
13                 Ride,
14                 CheckInfo,
15                 Publish
16 }
17
18 var sig Trip {
19         var id: one idTrip,
20         var ComposedOf: one Path,
21         var BelongingTo: one System,
22         var userRelated: one user,
23         var situation: one currentSituation
24 } {
25         #ComposedOf = 1
26 }
27
28 sig user {
29         id: one idUser,
30         var tripInfo: lone Trip
31 } {
32 }
33
34 sig userAnonymous extends user {
35 }
36
37 sig Path {
38         id: idPath,
39         var PartOf: set Trip,
40         Recorded: one Way,
41         computedStatus: one Status
42 } {
43 }
44
45 one sig System {
46         ProvidingPaths: set Path,
47         toUser: some user
48 }{
49 }
50
51 ------------------------------ facts on step progression ------------------------------
52
53
54 // Every situation is a ordered step
55 fun nextSituation[t: Trip]: one currentSituation {
56         (t.situation = Search) => Select else
57         (t.situation = Select) => Ride else
58         (t.situation = Ride) => CheckInfo else
59         (t.situation = CheckInfo) => Publish else
60         (t.situation = Publish) => Search else none
61 }
62
63 // Recall nextSituation function in order to move along steps
64 fact stateProgression {
65         always (
66                 all t: Trip |
67                                 t.situation' = t.situation or
```

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

```
68                                  t.situation' = nextSituation[t] or
69                                  t not in Trip'
70          )
71  }
72
73  // Every steps appears at select
74  fact insertingTrip {
75          always (
76                  all t: Trip |
77                          once (t.situation = Select)
78          )
79  }
80
81  // Trip rules along steps
82  fact finalizingTrip {
83          always (all t: Trip |  after t.situation = CheckInfo implies eventually (t not in
        Trip))
84          always (all t: Trip | t not in Trip' implies after always t not in Trip')
85  }
86
87  // Trip can't disappear if appears at least once after select
88  fact persistencyAlongSteps {
89    all t: Trip| all cS: t.situation |
90      (cS = Search or cS = Select or cS = Ride or cS = CheckInfo ) implies
91        always (t in Trip')
92  }
93
94  fact persistencyPathAlongSteps {
95          all t: Trip, s: System | s in t.BelongingTo implies always (s in t'.BelongingTo)
96          all t: Trip, u: user | u in t.userRelated implies always (u in t'.userRelated)
97          all t: Trip, p: Path | p in t.ComposedOf implies always (p in t'.ComposedOf)
98          all t: Trip, idt: idTrip | idt in t.id implies always (idt in t'.id)
99  }
100
101 fact userPersistency {
102         user = user'
103 }
104
105 fact userOfTripRemainsSame {
106     always all t: Trip | t'.userRelated = t.userRelated
107 }
108
109 // all user have a unique email address and a unique username
110 fact uniqueID{
111         always ( all disj u1 , u2 : user | u1.id != u2.id )
112 }
113
114 // every Email is associated to one User
115 fact uniqueID {
116         always ( all i : idUser | one u : user | u.id = i )
117 }
118 ------------------------------------------------------------------------------------------------------


119
120 // Relationship between Trip, path and userAnonymous
121 fact {
122         no u: userAnonymous | all t: Trip | all p: Path | p in t.ComposedOf and t in u.
        tripInfo
123 }
124
125 fact {
126         all u: user | all s: System | u in s.toUser
127 }
128
129 fact {
130         all t: Trip | all u: user | u in t.userRelated implies (t in u.tripInfo)
131 }
132
133 fact {
134         all i: idUser | all u: user | all u2: user | i in u.id and u != u2 implies always
        (i not in u2.id)
135 }
```

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

```
136
137 // users must appear and be related to trip
138 fact  {
139         all t: Trip | all u: user | u = t.userRelated implies (no t2: Trip | t2 != t and
        u in t2.userRelated)
140         all t: Trip | all u: user | u = t.userRelated implies (no t2: Trip' | t2 != t and
         u in t2.userRelated)
141         all t: Trip | all u: user | u = t.userRelated implies after always (no t2: Trip |
         t2 != t and u in t2.userRelated)
142         all u: user | all t: Trip | u = t.userRelated implies always (t in u.tripInfo)
143 }
144
145 fact  {
146         always( all u1: user | all t: Trip | all s: System | t in u1.tripInfo implies (u1
         in t.userRelated and u1 in s.toUser and t.ComposedOf in s.ProvidingPaths))
147 }
148
149 fact {
150         always(all disj t1, t2: Trip | t1.id != t2.id )
151 }
152
153 fact {
154     all t: Path | t.id' = t.id
155 }
156
157 fact userTripHasPath {
158         always (all t: Trip | all u: user | t in u.tripInfo implies (all p: Path | p in t
        .ComposedOf))
159 }
160
161 fact {
162         always(all an: userAnonymous | all u: user | an != u)
163 }
164
165 fact {
166         all u1, u2: user | all t: Trip | t in u1.tripInfo and u1.id != u2.id implies (t
        not in u2.tripInfo)
167 }
168
169 // user is quantifiable and unique
170 fact distintUser {
171         all u1, u2: user | u1.id != u2.id iff (u1 != u2)
172         all u1, u2: user | u1 != u2 implies after always u1 != u2
173 }
174
175 fact {
176         all t1, t2: Trip | t1 != t2 implies  t1.id != t2.id
177         all t1, t2: Trip | t1 != t2 implies after always t1.id != t2.id
178 }
179
180 // future steps must have same patters of previous steps
181 fact {
182         (all sys: System | all t: Trip | all u: user | sys in t.BelongingTo and u in t.
        userRelated and t in u.tripInfo and u in sys.toUser implies after always ( System' =
        t.BelongingTo and user' = t.userRelated and user' = sys.toUser and Trip' = u.tripInfo
        ))
183 }
184
185 ------------------------------------------------------------------------------------------------------------
186
187 // G1: Users can register and have a profile in BBP application.
188
189 assert differentUsersCannotBeSameUser {
190         all u1, u2: user | u1.id  != u2.id implies u1 = u2
191 }
192
193 assert userRegisteredNotAnonymous {
194         all u: user | all an: userAnonymous | u = user implies u != an
195 }
196
197 // G2: Users manually create bike paths in an inventory by specifying the name of the
```

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

```
    streets and the status.
198
199 pred showPath {
200 #Trip = 0
201 eventually (#Trip = 2)
202 }
203
204 // G3: Users create bike paths in an automated mode where the system acquire data from
    the user's mobile device and reconstruct the path.
205
206 assert AutomatedXorManual {
207         all w: Way | all p: Path | w = Automated and w =  p.Recorded implies (w != Manual
    )
208         all w: Way | all p: Path | w = Manual and w = p.Recorded implies (w != Automated)
209 }
210
211 // G4: Users browse bike paths within the inventory. They can specify an origin and
    destination and ask the system to visualize the bike path on a map.
212
213 fun searchPath[t: Trip]: set Path{
214         {
215                 p: Path | some g: t.ComposedOf | p = g
216         }
217 }
218
219 pred selectPath[t: Trip]{
220         some searchPath[t]
221 }
222
223 // G5: Users record their trips and store them to keep track of their cycling activities.
224
225 fun extractTrip[u: user]: lone Trip {
226     u.tripInfo
227 }
228
229 pred viewTrip [u: user] {
230         #Trip = 0
231         eventually (#Trip = 1)
232         #extractTrip[u] <= 1
233 }
234
235 // G6: Users can see for a recorded trip various statistical data of the user
    performances as well as meteorological information during the respective trip.
236
237 pred viewTripAtCheckInfo[u: user] {
238     #Trip = 0
239         eventually ( some t:Trip | #Trip = 1 and t.situation = CheckInfo and  t in u.
    tripInfo )
240 }
241
242 // G7: The system uses information published by the users about the paths by merging
    pieces of information to generate the optimal path, using the freshness and number of
     confirmed data.
243
244 pred show[] {
245     #Trip = 0
246     eventually (#Trip = 3)
247     eventually (some t: Trip | t.situation = Publish)
248 }
249
250 //check differentUsersCannotBeSameUser for 2 but exactly 3 user
251 //check userRegisteredNotAnonymous  for 3 but exactly 3 user
252 //check AutomatedXorManual for 2
253 //run viewTripAtCheckInfo for 3 but exactly 3 user
254 //run showPath for 3 but exactly 3 user
255 //run viewTrip for 3 but exactly 3 user
256 //run selectPath for 3 but 1 Path, 5 steps
257 //run show for 3 but exactly 3 user
```

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

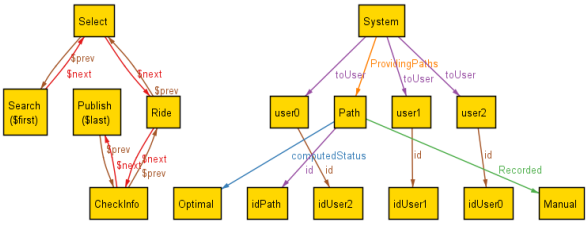


Figure 25: Idle phase

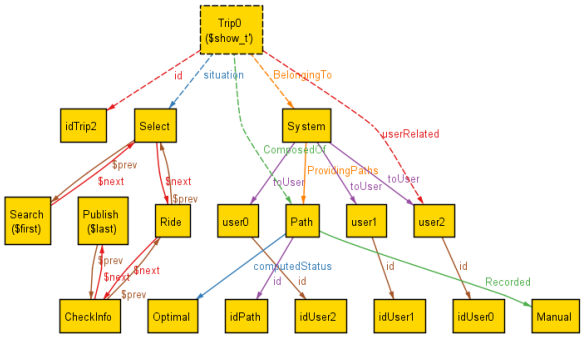


Figure 26: Select Phase

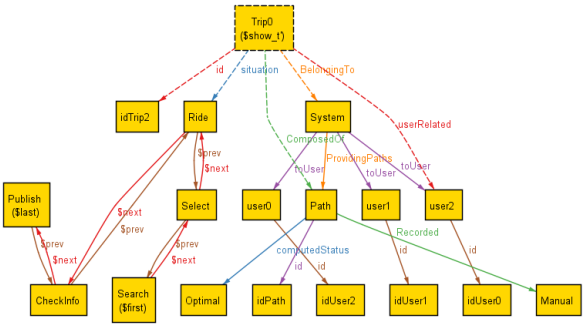


Figure 27: Ride phase

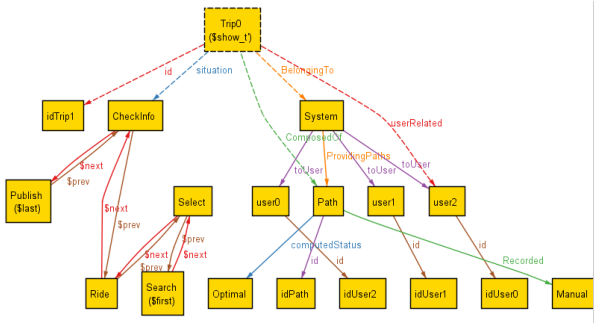


Figure 28: Checkinfo phase

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

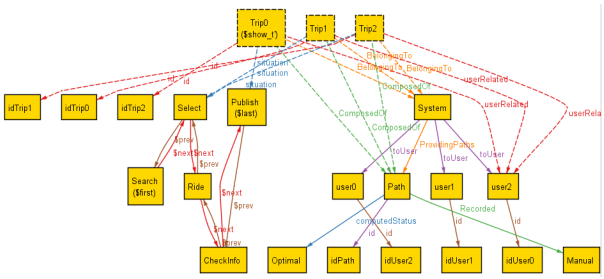


Figure 29: Publish phase

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

5 Effort Spent

The following table displays the number of hours each group member spent on the various sections of the document.

Group members	Chapter 1	Chapter 2	Chapter 3	Chapter 4
Alessandro Barbera	2	8	22	1
Nicolas Bolanos	4	11	8	12
Massimiliano Nuzzo	1	3	5	25

6 Use of Generative AI

6.1 Use of Generative Artificial Intelligence Tools

Generative Artificial Intelligence (AI) tools were employed to support various stages of the documentation and design process in this project. The tools used include **Perplexity AI** and **Gemini**, selected for their advanced natural language generation and reasoning capabilities. These tools assisted in tasks related to technical writing, software modeling, and language revision, always under careful human supervision and validation.

All strategic and main decision processes were entirely human-driven. AI served strictly as a tool to augment these decisions, automate routine work, and save time, without influencing core architectural or methodological choices.

The specific prompts provided to the AI systems were explicitly formulated to focus on technical and linguistic support. Examples of inputs include:

- "Help me generate a PlantUML code Domain Class Diagram from the following Domain Class Diagram sketch."
- "Help me brainstorm goals for the following software use case."
- "Help me write in a formal way the following functional requirements."
- "Help me rewrite the following section for a software design document."
- "Help me look for grammar and spelling mistakes."

The outputs obtained included automatically generated textual content such as structured PlantUML code for all diagrams, rewritten and polished technical documentation, formalized versions of functional requirements, and identified language corrections. In brainstorming tasks, the tools provided diverse options and design considerations that served as inspiration for choosing what to include in the document.

Verification, refinement, and integration of AI-generated outputs were performed systematically. Each generated segment was carefully reviewed before inclusion. For code artifacts (Alloy), outputs were tested directly within the target environment or modeling software to confirm proper functionality and consistency with the project's requirements. When AI responses incorporated external knowledge sources, these references were verified for accuracy and reliability before use. Sections of text that did not meet quality or factual standards were either revised or discarded entirely.

The integration of AI assistance enhanced productivity and coherence across the project documentation. It supported the generation of consistent technical descriptions and facilitated the refinement of complex technical ideas into well-structured prose. Human oversight remained indispensable; all final content (including diagrams,

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

requirements, and textual documentation) was critically validated, edited, and approved by the authors to ensure academic rigor, correctness, and originality.

Overall, Generative AI tools functioned as an auxiliary mechanism for idea development, code drafting, and language improvement, rather than as a replacement for human analysis or authorship. Through responsible use and verification, these tools contributed to improving the clarity, precision, and quality of the final project deliverables.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

References

[1] Amazon Web Services. Amazon simple email service, 2025. URL: https://aws.amazon.com/ses/.

[2] Kirill Fakhroutdinov. Uml class and object diagrams overview, 2009-2025. URL: https://www.uml-diagrams.org/class-diagrams-overview.html.

[3] Kirill Fakhroutdinov. Uml state machine diagrams, 2009-2025. URL: https://www.uml-diagrams.org/state-machine-diagrams.html.

[4] Kirill Fakhroutdinov. Uml sequence diagrams, 2025. URL: https://www.uml-diagrams.org/sequence-diagrams.html.

[5] Kirill Fakhroutdinov. Uml use case diagrams, 2025. URL: https://www.uml-diagrams.org/use-case-diagrams.html.

[6] GDPR-Info.eu. General data protection regulation (GDPR) – legal text, 2025. URL: https://gdpr-info.eu.

[7] Google. Maps sdk for android, 2026. URL: https://developers.google.com/maps/documentation/android-sdk?hl=en.

[8] ISO. Iso/iec/ieee 12207:2017, 2017. URL: https://www.iso.org/standard/63712.html.

[9] ISO. Iso/iec 27001:2022, 2022. URL: https://www.iso.org/standard/27001.

[10] ISO. Iso/iec 25010:2023, 2023. URL: https://www.iso.org/standard/78176.html.

[11] OpenWeatherMap. Weather api, 2025. URL: https://openweathermap.org/api.

[12] Strava. Strava releases 12th annual year in sport trend report 2025, 2025. URL: https://press.strava.com/articles/strava-releases-12th-annual-year-in-sport-trend-report-2025.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved